

세션 3 · Kafka 운영

Broker · Topic · Partition · Producer · Consumer Group · Offset · 복제 · Retention

대상 버전

Kafka 4.3.x

OS

Ubuntu 24.04 LTS

실습 환경

Azure 한국 중부

01

인프라 구축

VNet · 서브넷

네트워크 보안 그룹

VM · 디스크

가용성 영역

ops 노드

03-kafka 스택

노드	사양	역할
kafka-1 · kafka-2 · kafka-3	Standard_D2ds_v5 2 vCPU / 8 GiB 데이터 디스크 256 GiB	Broker 노드
ops-1	Standard_D2ds_v5 2 vCPU / 8 GiB 데이터 디스크 64 GiB	명령 실행, Prometheus·Grafana

1일차 스택과 다른 것은 VM 계열과 디스크 용량 둘입니다.

1일차 서비스 노드는 vCPU당 8 GiB 인 E 계열이었고, 이 세션의 노드는 4 GiB 인 D 계열입니다. Kafka 는 메모리를 JVM 힙이 아니라 OS 캐시로 쓰기 때문 입니다.

변수 파일 작성

```
subscription_id      = "00000000-0000-0000-0000-000000000000"
resource_group_name  = "rkm-student07-kafka-rg"
location             = "koreacentral"
allowed_cidrs        = ["203.0.113.10/32"]
ssh_public_key_path  = "~/.ssh/rkm.pub"
```

- **resource_group_name** : rkm-<이름>-kafka-rg 형식. 이름이 겹치면 다른 수강생의 자원을 지우게 됩니다
- **allowed_cidrs** : 현재 PC 의 공인 IP 를 /32 로. 강의장 IP 가 바뀌면 다시 **apply** 합니다
- **ssh_public_key_path** : 1일차에 만든 키를 그대로 씁니다

이 절차는 자기 PC 에서 실행합니다.

ops 노드가 아닙니다. `cd lab/terraform/03-kafka` 후 `cp session.tfvars.example session.tfvars` 로 복사해 다섯 항목을 채웁니다.

스택 생성

- 1 `cd lab/terraform/03-kafka`
- 2 `terraform init`
- 3 `terraform plan -var-file=session.tfvars` 로 자원 수 확인
- 4 `terraform apply -var-file=session.tfvars`
- 5 `terraform output` 으로 ops 노드 공인 IP 확인

이 절차는 자기 PC 에서 실행합니다.

ops 노드는 아직 없습니다. 이 절차가 끝나야 만들어집니다.

완료 판정

`Apply complete!` 로 끝나고 출력 세 개에 값이 전부 채워지면 완료입니다.

생성되는 자원

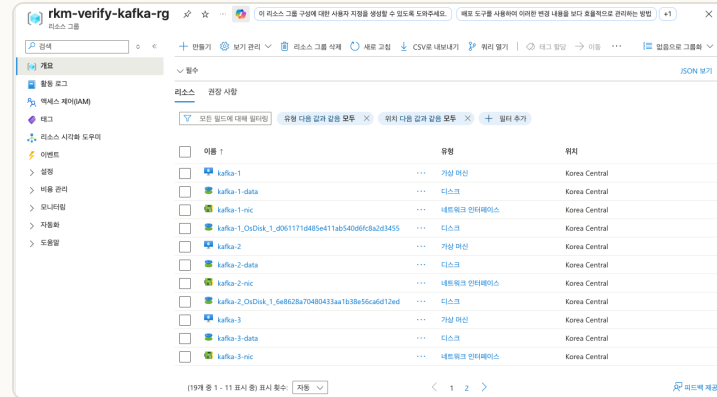


- **VM 4대:** Broker 3대와 ops 1대
- **가용성 영역:** kafka-1 · kafka-2 · kafka-3 이 영역 1·2·3에 하나씩
- **공인 IP:** ops-1 에만 붙습니다. Broker 에는 없습니다

포털 확인 항목

확인 대상	포털 메뉴	확인 항목
VM 크기	가상 머신	Standard_D2ds_v5 . 1일차 두 세션은 E 계열
VM과 영역	가상 머신	kafka-1 · kafka-2 · kafka-3 이 영역 1·2·3에 하나씩
디스크	디스크	256 GiB , 3,000 IOPS, 125 MB/s
인바운드 규칙	네트워크 보안 그룹	22·3000·9090이 현재 PC 공인 IP로 제한

생성 결과 확인



리소스 그룹의 자원 목록

+ 만들기 예약 보기 관리 새로 고침 CSV로 내보내기 쿼리 열기 태그 할당 시작 다시 시작 중지 삭제 서비스 없음으로 그룹화						
모든 필드에 대해 필터링 구독 다음 값과 같음 모두 유형 다음 값과 같음 모두 리소스 그룹 다음 값과 같음 모두 위치 다음 값과 같음 모두 필터 추가						
☐	이름 ↑	리소스 그룹	위치	상태	크기	가용성 영역
☐	kafka-1	... rkm-verify-kafka-rg	Korea Central	실행 중	Standard_D2ds_v5	1
☐	kafka-2	... rkm-verify-kafka-rg	Korea Central	실행 중	Standard_D2ds_v5	2
☐	kafka-3	... rkm-verify-kafka-rg	Korea Central	실행 중	Standard_D2ds_v5	3
☐	ops-1	... rkm-verify-kafka-rg	Korea Central	실행 중	Standard_D2ds_v5	1

VM 목록과 크기·가용성 영역

데이터 디스크 확인

개요

활동 로그

액세스 제어(IAM)

태그

문제 진단 및 해결

리소스 시각화 도구미

설정

구성

크기 + 성능

암호화

네트워킹

디스크 성능을 향상하는 방법 탐색

스토리지 유형 ⓘ

프리미엄 SSD v2(로컬 중복 스토리지) ▼

일부 옵션을 사용할 수 없는 이유는 무엇인가요? ⓘ

최대 크기	디스크 계층	프로비전된 IOPS	프로비전된 처리량	최대 공유 ⓘ	최대 버스트 IOPS ⓘ	최대 버스트 처리량
65536 GiB	P	80000	2000	15	-	-

사용자 지정 디스크 크기(GiB) * ⓘ

256

디스크 IOPS *

3000 ✓

디스크 처리량(MB/초) *

125 ✓

데이터 디스크의 용량과 IOPS·처리량

- **용량 256 GiB:** Kafka 가 메시지를 쌓는 로그 파일 전용. OS 디스크와 별도로 붙는 관리 디스크
- **IOPS 3,000 · 처리량 125 MB/s:** Premium SSD v2 의 무료 최저값

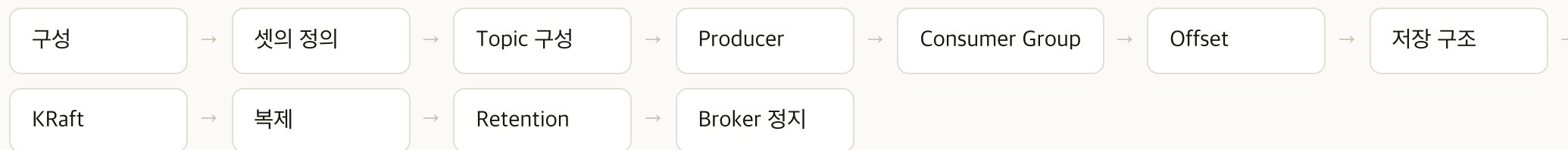
세션 2 의 데이터 디스크는 128 GiB 였습니다. 로그 파일이 여러 개로 나뉘어 쌓이는 것을 보려고 두 배로 잡았습니다.

ops 노드 접속

```
# Git Bash
ssh -i ~/.ssh/rkm azureuser@<ops 공인 IP>
```

- **접속 대상:** terraform output 의 ops_public_ip
- **Broker 접속:** ops 노드 안에서 ssh 10.0.1.5 로 들어갑니다. 키는 부팅 때 배치됩니다
- **이후 모든 명령:** ops 노드에서 실행합니다

세션 진행 순서



기본 개념과 데이터 경로를 먼저 다루고 내부 구조를 뒤에 둡니다.

Broker·Topic·Partition 다음에 메시지를 넣고 빼는 실습까지 다루고, 로그 저장 구조와 KRaft·복제는 그 뒤에 둡니다.

02

Kafka 구성

인벤토리

클러스터 ID

스토리지 포맷

JVM 힙

listeners

관측 스택

구성 단계의 산출물

role	하는 일	왜 필요한가
common	데이터 디스크 마운트, 시간 동기화	로그 파일을 OS 디스크와 분리합니다
common	파일 핸들 한도 상향	Partition 마다 로그 파일이 여러 개 열립니다
kafka	Kafka 4.3.x 설치와 <code>server.properties</code> 배포	세 노드가 같은 설정으로 기동합니다
kafka	클러스터 ID 생성과 스토리지 포맷	KRaft 는 메타데이터를 자신의 로그에 담습니다
kafka	JVM 힙 2 GiB	남은 메모리를 OS 캐시로 넘깁니다
monitoring	exporter 와 Prometheus·Grafana	이 세션에서 바로 확인합니다

이 단계에서 Broker 3대가 KRaft 클러스터 하나로 묶입니다. Topic 은 아직 없고 챕터 04 실습에서 만듭니다.

인벤토리 노드 목록

```
[kafka]
kafka-1 ansible_host=10.0.1.5 kafka_node_id=1
kafka-2 ansible_host=10.0.1.6 kafka_node_id=2
kafka-3 ansible_host=10.0.1.7 kafka_node_id=3

[ops]
ops-1 ansible_connection=local

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa

kafka_client_port=9092
kafka_controller_port=9093
```

- 채우는 주체: Terraform. 사설 IP를 채운 파일을 ops 노드에 배치
- 받는 시점: ops 노드가 부팅할 때 cloud-init 이 저장소 사본 위에 덮어쓰
- **[ops]** : **local** 연결. 구성을 돌리는 노드 자신

server.properties 주요 설정

```
process.roles=broker,controller
node.id={{ kafka_node_id }}
controller.quorum.voters={{ kafka_quorum_voters }}
controller.listener.names=CONTROLLER
log.dirs={{ kafka_log_dir }}
```

- **process.roles** : broker,controller . 한 프로세스가 둘을 겸합니다
- **node.id** : 노드마다 다릅니다. 클러스터 안에서 유일해야 합니다
- **log.dirs** : 데이터 디스크의 마운트 지점입니다. OS 디스크가 아닙니다

controller.quorum.voters 와 **node.id** 가 어긋나면 기동 직후 종료합니다.
세 노드의 ID 가 voters 목록에 그대로 들어 있어야 합니다.

advertised.listeners 를 사실 IP 로 두는 근거

항목	내용
무엇인가	Broker 가 클라이언트에게 알려 줄 주소입니다. 접속을 받는 주소(listeners)와 다릅니다
어떻게 쓰이는가	클라이언트는 <code>--bootstrap-server</code> 로 한 번 접속한 뒤, 응답에 담긴 주소로 다시 붙습니다
왜 문제인가	여기에 호스트 이름이 들어가면 ops 노드가 그 이름을 해석하지 못해 챕터 04 실습부터 진행되지 않습니다
어떻게 푸는가	사실 IP 를 그대로 적어 배포합니다
안전한가	Broker 에 공인 IP 가 없고, 네트워크 보안 그룹이 서브넷 안에서 오는 것만 허용합니다

`--bootstrap-server` 는 첫 접속에만 쓰입니다.

실제 통신 주소는 Broker 가 알려 줍니다. 첫 접속은 되는데 그 뒤가 되지 않으면 이 설정을 봅니다.

클러스터 ID 와 스토리지 포맷

항목	내용
왜 필요한가	KRaft 는 메타데이터를 Kafka 자신의 로그에 담습니다. 그 로그를 쓰려면 스토리지를 먼저 초기화해야 합니다
무엇을 하는가	<code>kafka-storage.sh format</code> 이 <code>log.dirs</code> 에 <code>meta.properties</code> 를 만듭니다
언제 하는가	최초 1회만 합니다. 이미 포맷된 디렉터리에 다시 돌리면 실패합니다
세 노드가 같아야 함	클러스터 ID 를 한 번 만들어 세 노드에 같은 값을 넣습니다
다르면	Broker 가 서로를 받아들이지 않고 클러스터가 서지 않습니다

포맷은 그 디렉터리의 데이터를 없앤 상태에서 출발합니다.

이미 데이터가 있는 Broker 에 다시 돌리지 않습니다. 이 실습에서는 첫 구성에서 한 번만 실행합니다.

JVM 힙을 2 GiB 로 두는 근거

항목	배분
OS + exporter	1 GiB
JVM 힙	2 GiB
OS 캐시	약 5 GiB

- **Kafka 는 메시지를 힙에 담지 않습니다:** 디스크에 쓰고 OS 캐시에서 읽습니다
- **힙을 늘리면 캐시가 줄어듭니다:** 읽기가 디스크로 내려가 지연이 커집니다
- **권장 상한:** 6 GiB 이하입니다. 그보다 크면 GC 정지가 길어집니다

균형형 D 계열이 이 배분에 맞습니다.

vCPU당 8 GiB 인 E 계열은 힙을 작게 두는 Kafka 에 과합니다.

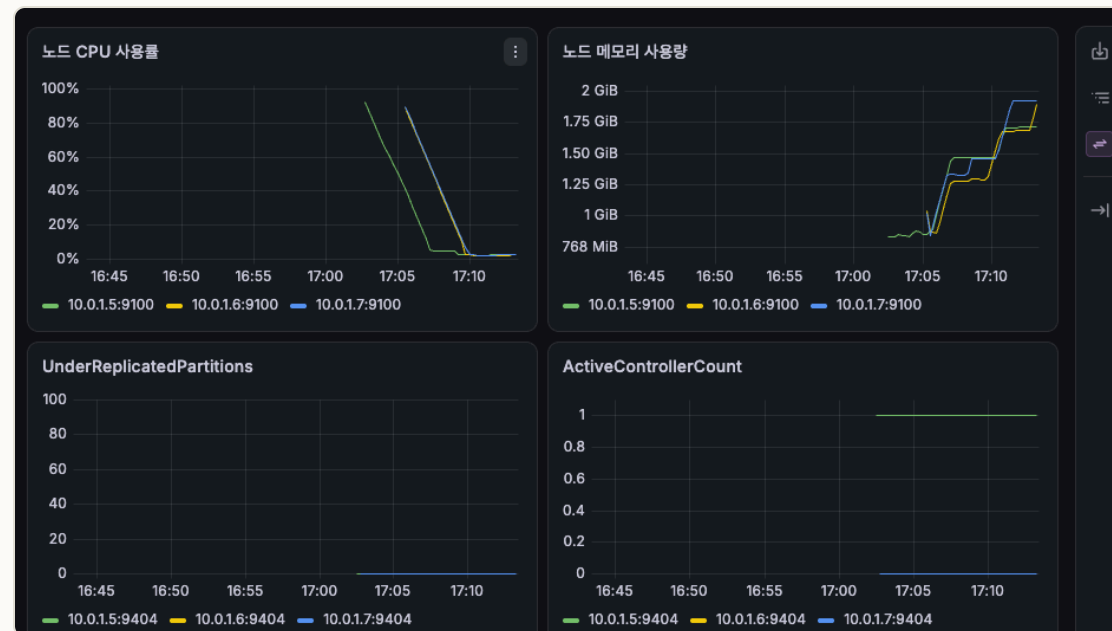
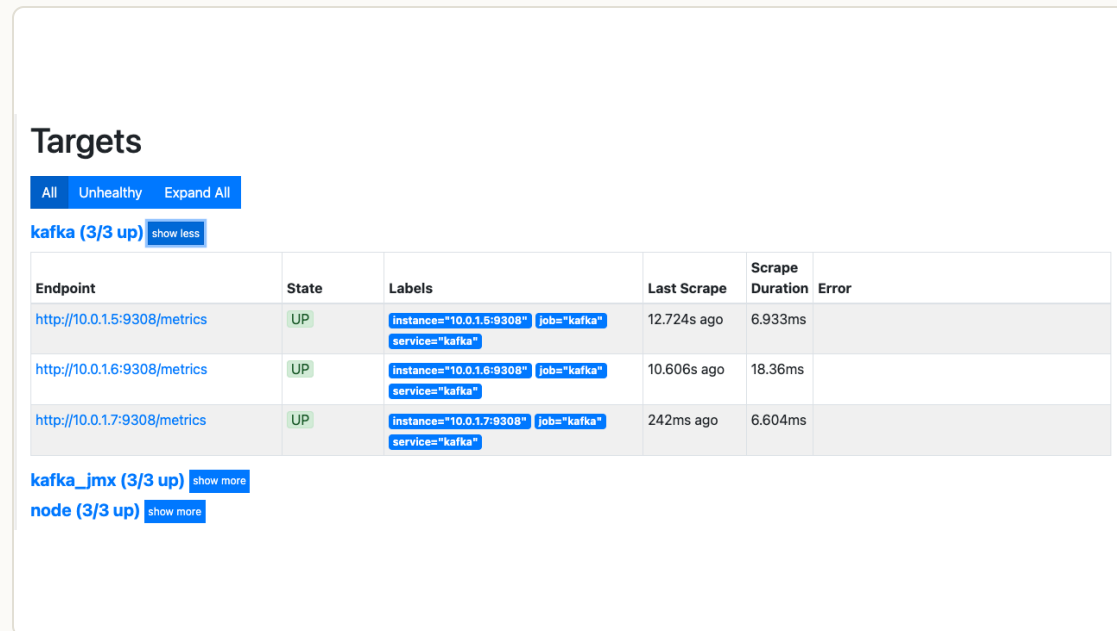
구성 실행

- 1 `cd ~/ansible`
- 2 `ansible kafka -m ping` 으로 Broker 3대 연결 확인
- 3 `ansible-playbook play-kafka.yml` 실행
- 4 `ansible kafka -a "systemctl is-active kafka"` 로 기동 확인
- 5 `ansible kafka -a "grep cluster.id /var/lib/kafka/meta.properties"` 로 세 노드의 클러스터 ID 확인
- 6 `source /etc/profile.d/kafka.sh` 로 지금 셸에 Kafka CLI 경로 적용

PLAY RECAP

```
kafka-1  : ok=42  changed=33  unreachable=0  failed=0  skipped=17
kafka-2  : ok=42  changed=32  unreachable=0  failed=0  skipped=17
kafka-3  : ok=42  changed=32  unreachable=0  failed=0  skipped=17
ops-1    : ok=19  changed=15  unreachable=0  failed=0  skipped=26
```

관측 스택 확인



Grafana 대시보드

- **node** : 각 노드의 9100. CPU·메모리·디스크·네트워크
- **kafka** : 각 노드의 9308. Topic·Partition·Consumer Group 지표
- **kafka_jmx** : 각 노드의 9404. 복제 상태와 controller 상태처럼 Broker 프로세스 안에서만 나오는 값

여기서는 지표가 보이는지까지만 확인합니다.

Broker 접속 확인

```
# ops 노드에서
kafka-broker-api-versions.sh --bootstrap-server 10.0.1.5:9092 | grep "id:"

10.0.1.7:9092 (id: 3 rack: null isFenced: false) -> (
10.0.1.6:9092 (id: 2 rack: null isFenced: false) -> (
10.0.1.5:9092 (id: 1 rack: null isFenced: false) -> (
```

- 세 줄이 나와야 합니다: 한 Broker 에 접속했는데 셋이 다 보이는 것이 정상입니다
- **id** : `node.id` 입니다. 인벤토리에 적은 값과 같아야 합니다

한 Broker 만 지정해도 클러스터 전체가 보입니다.

첫 접속의 응답에 나머지 Broker 의 주소가 함께 담기기 때문입니다. `advertised.listeners` 가 그 주소입니다.

03

Broker·Topic·Partition

셋의 정의

배치

병렬성

순서 보장

키 파티셔닝

Partition 수

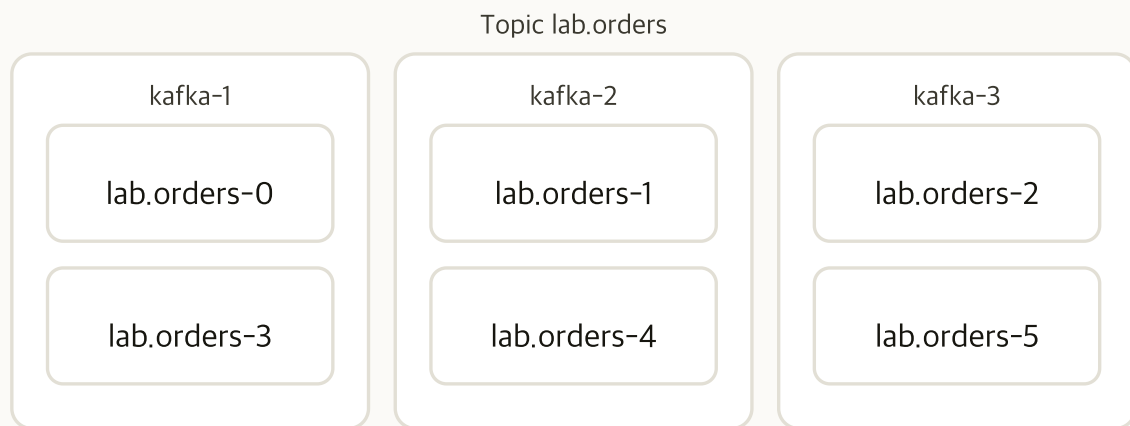
Broker · Topic · Partition

대상	무엇인가	물리인가 논리인가
Broker	Kafka 프로세스 하나이자 노드 하나. Partition 을 디스크에 들고 클라이언트 요청을 받습니다	물리. 프로세스와 디스크가 있습니다
Topic	메시지에 붙는 이름표. Producer 와 Consumer 가 이 이름으로 주고받습니다	논리. 실체가 없고 Partition 의 묶음입니다
Partition	실제로 디스크에 놓이는 로그. segment 파일의 연속입니다	물리. 디렉터리 하나가 Partition 하나입니다

Topic 에는 파일이 없습니다.

Broker 의 `log.dirs` 아래에 있는 것은 `lab.orders-0` 처럼 Partition 디렉터리입니다. Topic 은 그 묶음의 이름입니다.

Broker 와 Partition 의 배치



- **Topic 하나가 Partition 여섯으로 나뉩니다:** Partition 번호는 0부터입니다
- **Partition 이 Broker 에 흩어집니다:** 한 Broker 에 두 개씩 놓였습니다
- **Partition 하나는 쪼개지지 않습니다:** 통째로 한 Broker 에 들어갑니다

Topic 에 쓴다는 것은 그중 한 Partition 에 쓴다는 뜻입니다.
어느 Partition 으로 갈지는 키가 정합니다.

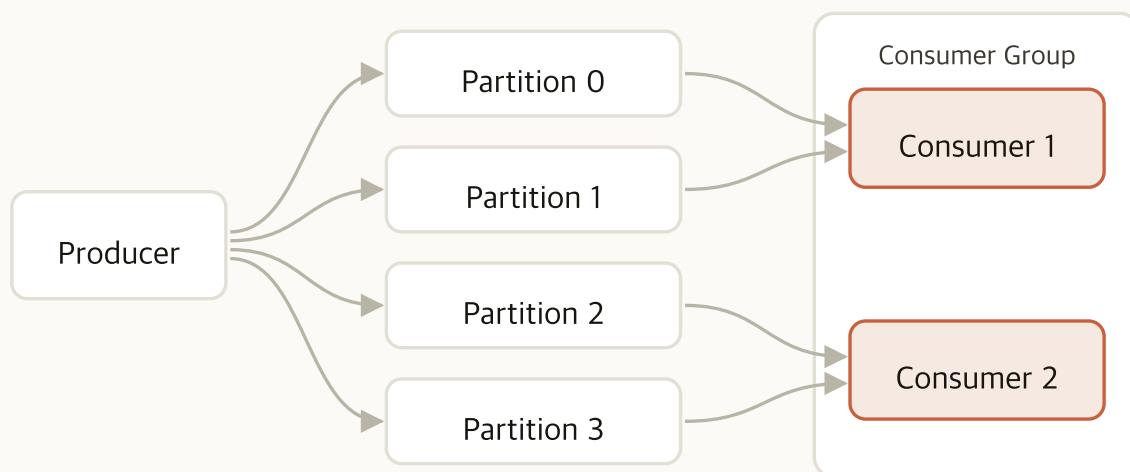
셋의 관계

관계	내용
Topic 과 Partition	Topic 하나가 Partition 여럿으로 나뉩니다. Topic 단위로 늘릴 수 있는 것은 Partition 수뿐입니다
Partition 과 Broker	Partition 하나가 Broker 하나에 통째로 들어갑니다. Broker 를 늘려도 기존 Partition 이 저절로 옮겨 가지 않습니다
Partition 수와 Broker 수	Partition 이 Broker 보다 많아도 됩니다. 그때 한 Broker 가 Partition 여럿을 맡습니다
Partition 과 leader	Partition 마다 읽기와 쓰기를 맡는 Broker 가 하나 정해집니다. 이것을 leader 라고 합니다

성능을 정하는 것은 Topic 수가 아니라 Partition 수입니다.

Topic 을 늘려도 병렬성이 오르지 않습니다. 같은 데이터를 더 나눠 처리하려면 Partition 을 늘립니다.

병렬성의 단위인 Partition



- **Partition 하나는 Consumer 하나가 읽습니다:** 그룹 안에서 겹치지 않습니다
- **Consumer 를 늘리면 나눠 갖습니다:** Partition 수까지입니다
- **Partition 보다 많이 띄우면 남습니다:** 남는 Consumer 는 아무것도 할당받지 못합니다

처리량의 상한은 Partition 수가 정합니다.

Consumer 를 아무리 늘려도 Partition 수를 넘지 못합니다. 챕터 06 실습에서 직접 확인합니다.

메시지 순서 보장

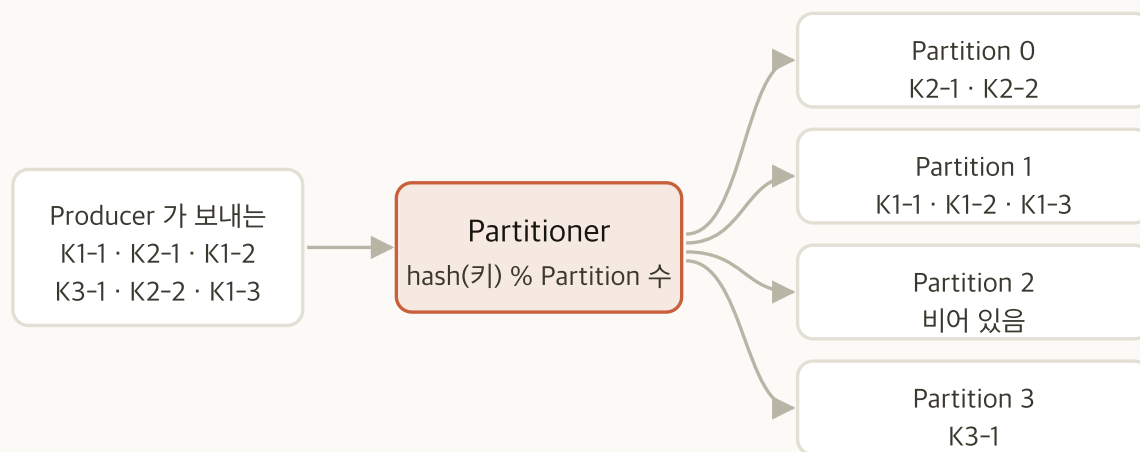


- **Partition 안에서만 보장됩니다:** 같은 Partition 에 들어간 메시지는 넣은 순서 그대로 읽힙니다
- **Partition 이 다르면 보장이 없습니다:** 먼저 보낸 것이 먼저 읽힌다는 보장이 없습니다
- **전역 순서가 필요하면 Partition 1개입니다:** 그러면 Consumer 도 하나여서 확장이 되지 않습니다
- **실무의 해법은 키입니다:** 순서가 필요한 단위를 키로 잡아 같은 Partition 으로 보냅니다

「전체 순서」가 아니라 「무엇의 순서가 필요한가」를 먼저 정합니다.

주문 하나의 상태 변화 순서가 필요한 것이지, 서로 다른 주문 사이의 순서가 필요한 경우는 드뭅니다.

키 기반 Partition 배정



- 키 해시를 Partition 수로 나눈 나머지: 기본 Partitioner 의 동작입니다
- 같은 키는 늘 같은 Partition 으로: 그래서 그 키 안에서는 순서가 보장됩니다
- 키가 없으면 고르게 흩어집니다: 순서 보장이 없는 대신 분배가 고릅니다

키를 잡는 것이 곧 순서 보장 단위를 정하는 것입니다.

주문 ID 를 키로 잡으면 주문 하나의 이벤트 순서가 지켜지고, 서로 다른 주문은 병렬로 처리됩니다.

Partition 수 변경의 영향

구분	Partition 3개	Partition 4개
A	<code>hash(A) % 3</code>	<code>hash(A) % 4</code> . 결과가 달라집니다
B	<code>hash(B) % 3</code>	<code>hash(B) % 4</code>
결과	각 키가 늘 같은 자리	같은 키가 다른 Partition 으로 갑니다

Partition 수를 늘리면 기존 키의 순서 보장이 깨집니다.

같은 키의 옛 메시지는 옛 Partition 에, 새 메시지는 새 Partition 에 있습니다. 두 Partition 사이에는 순서가 없습니다.

되돌릴 수도 없습니다.

Partition 수를 줄이는 명령이 없습니다. 되돌리려면 Topic 을 새로 만들고 데이터를 옮겨야 합니다.

Partition 수 결정

- **처리량 기준:** 목표 처리량을 Partition 하나가 감당하는 처리량으로 나눕니다
- **Consumer 기준:** 앞으로 띄울 Consumer 수 이상이어야 합니다. 그보다 적으면 Consumer 가 할당을 받지 못합니다
- **Broker 기준:** Broker 수의 배수로 잡으면 leader 가 고르게 나뉩니다
- **여유:** 늘리는 대가가 크므로 처음에 조금 넉넉히 잡습니다

앞의 세 기준 중 가장 큰 값을 씁니다.

처리량으로는 2개면 되는데 Consumer 를 6개 띄울 계획이면 6개입니다.

과다한 Partition 의 대가

자원	왜 늘어나는가	무엇이 걸리는가
파일 핸들	Partition 마다 segment 파일과 인덱스 파일이 열립니다	OS 한도에 걸려 Broker 가 파일을 못 엽니다
메타데이터	Partition 마다 leader·복제본 정보가 있습니다	controller 의 로그가 커지고 전파가 느려집니다
leader 선출 시간	Broker 가 정지하면 그 Broker 가 leader 였던 Partition 을 전부 다시 뽑습니다	Partition 이 많을수록 복구가 깁니다
Producer 메모리	Partition 마다 배치 버퍼를 잡습니다	클라이언트 메모리가 늡니다

근거 없이 크게 잡지 않습니다.

늘리는 것이 어렵다고 처음부터 수천 개로 잡으면 그 대가를 계속 치릅니다.

04

Topic 구성

Topic 생성

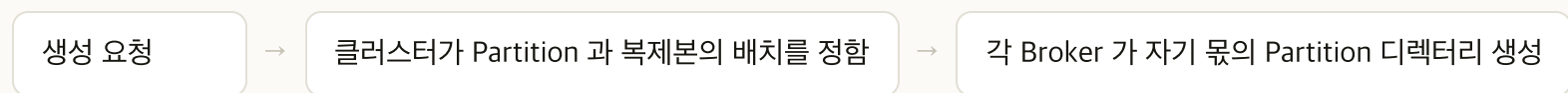
Partition 배치

복제본 배치

키 파티셔닝

Partition 수 변경

Topic 생성의 실체



- RF (`replication.factor`): Partition 하나를 몇 개의 Broker 에 복사해 둘 것인가입니다
- Partition 수와 RF 를 함께 정합니다: 만든 뒤에 Partition 은 늘릴 수만 있습니다
- RF 는 Broker 수를 넘을 수 없습니다: 복제본이 서로 다른 Broker 에 놓이므로 놓을 자리가 없습니다
- 배치를 사람이 지정하지 않습니다: Partition leader 가 Broker 에 고르게 나눕니다

Topic 을 만드는 것은 디렉터리를 만드는 것입니다.

Partition 마다 디렉터리 하나가 Broker 의 `log.dirs` 아래에 생깁니다.

실습 Topic 생성

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.orders --partitions 6 --replication-factor 3`
- 2 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders` 로 배치 확인
- 3 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.rf4 --partitions 1 --replication-factor 4` 로 거부되는 것 확인

완료 판정

`lab.orders` 여섯 Partition 의 `Leader` 가 세 Broker 에 고르게 나뉘고 `Replicas` 가 셋씩이며, `RF=4` 생성이 거부되면 완료입니다.

`lab.orders` 의 Partition 수를 이 세션 안에서 바꾸지 않습니다.

챕터 06 Consumer 실습이 6개를 전제로 합니다.

--describe 출력

```
Topic: lab.orders TopicId: yaCgIvGp... PartitionCount: 6 ReplicationFactor: 3
Configs: min.insync.replicas=1
Topic: lab.orders Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3 Elr: LastKnownElr:
Topic: lab.orders Partition: 1 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1 ...
Topic: lab.orders Partition: 2 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2 ...
... Partition 3·4·5 가 같은 방식으로 이어집니다
```

열	뜻	판정
Leader	그 Partition 의 읽기·쓰기를 맡는 Broker	세 Broker 에 두 개씩 고르게 나뉘어야 합니다
Replicas	복제본을 두는 Broker 목록. 첫 값이 선호 leader 입니다	셋이어야 합니다
Isr	leader 를 따라잡은 복제본	지금은 셋이 다 들어 있으면 됩니다

RF=4 가 거부되는 이유

```
Error while executing topic command : Unable to replicate the partition 4 time(s):
The target replication factor of 4 cannot be reached because only 3 broker(s)
are registered or some brokers have all their log directories cordoned.
```

- 복제본은 서로 다른 Broker 에 놓입니다: 같은 Broker 에 둘을 두면 그 Broker 가 정지할 때 둘 다 잃습니다
- 그래서 RF 상한이 Broker 수입니다: 3대 클러스터에서 RF 는 3까지입니다

RF 를 Broker 수와 같게 두어도 Broker 를 늘릴 때 RF 가 따라 늘지는 않습니다.

새 Broker 에 복제본을 놓으려면 Partition 재배포를 따로 실행합니다. 이 세션의 범위 밖입니다.

키와 Partition 확인

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.keys --partitions 3 --replication-factor 3`
- 2 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --reader-property parse.key=true --reader-property key.separator=:` 실행 후 `A:1` `D:1` `F:1` `A:2` `D:2` 를 한 줄씩 입력하고 종료
- 3 `for i in 0 1 2; do echo "-- partition $i --"; kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --partition $i --from-beginning --formatter-property print.key=true --timeout-ms 3000; done`
- 4 어느 키가 어느 Partition 에 있는지 기록

완료 판정

`A` 의 두 메시지가 같은 Partition 에 있고, 세 키가 서로 다른 Partition 에 나뉘어 있으면 완료입니다.

Partition 별 내용

```
-- partition 0 --
D    1
D    2
-- partition 1 --
A    1
A    2
-- partition 2 --
F    1
```

- **A** 의 두 건이 같은 Partition 에 있습니다: 같은 키는 같은 Partition 입니다
- Partition 안에서는 넣은 순서 그대로입니다: **1** 다음에 **2** 입니다
- 키가 Partition 에 고르게 나뉘지는 않습니다: 해시 결과이므로 몰릴 수 있습니다

Partition 수 증가의 영향

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --alter --topic lab.keys --partitions 4` 로 Partition 을 4개로
- 2 p.37 의 2번을 다시 실행해 `A:3` `D:3` `F:3` 입력
- 3 p.37 의 3번을 Partition 0부터 3까지로 넓혀 다시 실행
- 4 `A` 의 3번째 메시지가 앞의 둘과 다른 Partition 에 있는지 확인
- 5 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --alter --topic lab.keys --partitions 2` 로 줄이는 시도. **거부됩니다**

이 실습은 `lab.keys` 에서만 합니다.

`lab.orders` 의 Partition 수를 바꾸면 챕터 06 Consumer 실습이 성립하지 않습니다. 그리고 되돌릴 수 없습니다.

Partition 수를 줄일 수 없는 이유

```
Error while executing topic command : The topic lab.keys currently has
  4 partition(s); 2 would not be an increase.
```

- **줄이는 명령이 없습니다:** 없앨 Partition 의 메시지를 어디로 옮길지 정할 방법이 없습니다
- **되돌리려면:** Topic 을 새로 만들고 데이터를 옮깁니다

Partition 수는 한 방향으로만 갑니다.

처음에 정할 때 Consumer 계획까지 보고 잡는 이유가 이것입니다.

05

Producer

전송 경로

배치

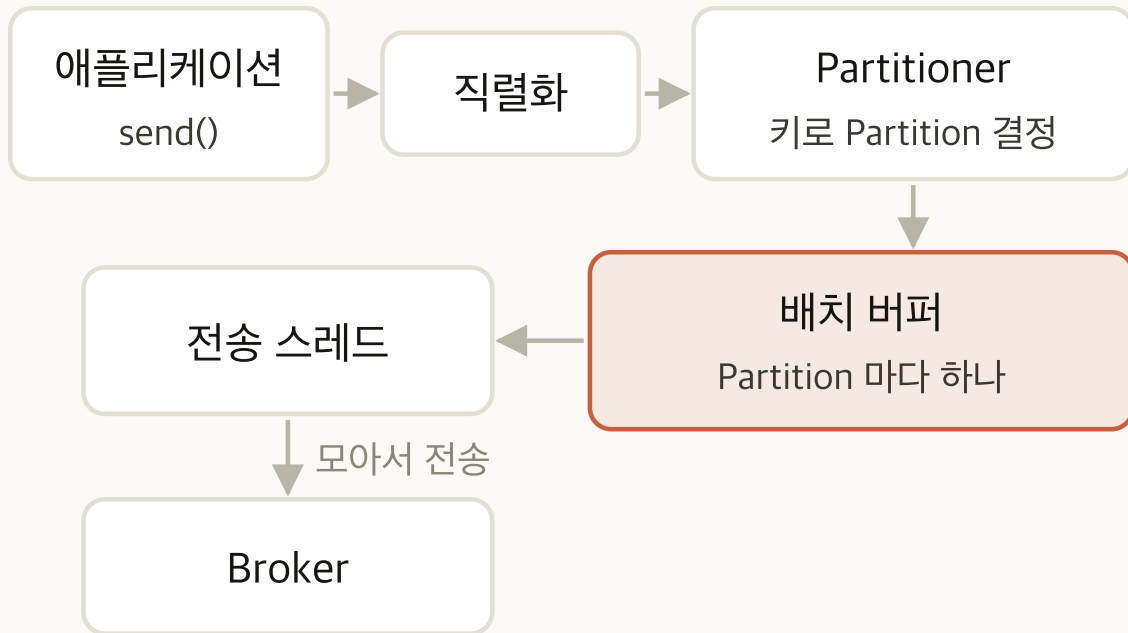
먹등성

압축

순서

재시도

Producer 의 전송 경로



- **send()** 는 바로 보내지 않습니다: 버퍼에 넣고 돌아옵니다
- 버퍼는 Partition 마다 있습니다: Partition 이 많으면 버퍼도 많아집니다
- 전송 스레드가 모아 보냅니다: 이 지점이 처리량과 지연을 가릅니다

send() 가 돌아온 것은 보낸 것이 아닙니다.

Broker 가 받았는지는 Broker 응답으로 확인합니다.

Producer 배치 설정

Producer 는 `send()` 를 받을 때마다 보내지 않고 Partition 마다 있는 배치 버퍼에 쌓아 두었다가 모아서 보냅니다. 언제 보낼지를 정하는 것이 아래 두 값입니다.

설정	무엇인가	올리면
<code>batch.size</code>	Partition 하나의 배치가 이 크기에 차면 곧바로 보냅니다	한 번에 더 많이 보냅니다. 처리량이 오릅니다
<code>linger.ms</code>	배치가 다 차지 않았을 때 더 모으려고 기다리는 최대 시간입니다. 이 시간이 지나면 덜 찼어도 보냅니다	배치가 커져 처리량이 오릅니다. 대신 그만큼 지연이 늘니다
<code>buffer.memory</code>	Partition 별 배치 버퍼를 전부 합쳐 쓸 수 있는 메모리 총량입니다	Broker 가 느릴 때 더 오래 쌓아 둘 수 있습니다

`linger.ms=0` 은 실시간 설정이 아니라 「기다리지 않음」입니다.

배치에 들어 있는 것을 바로 보내므로 지연이 가장 짧습니다. 대신 배치가 거의 모이지 않아 요청 수가 늘고 처리량이 가장 낮습니다.

enable.idempotence

상황	Producer 동작	결과
멥등성 꺼짐 · Broker 가 받고 응답 직전에 끊김	재시도	같은 메시지가 두 번 저장됩니다
멥등성 켜짐 · 같은 상황	Producer ID 와 시퀀스 번호를 함께 보냄	Broker 가 중복을 걸러냅니다

- **기본 활성화입니다:** 3.0 부터 기본값이 `true` 입니다
- **끄면 재시도가 중복을 만듭니다:** 네트워크가 불안한 곳에서 바로 드러납니다
- **Partition 단위입니다:** Partition 하나 안에서 중복과 순서를 보장합니다

재시도는 기본 동작입니다.

끄지 않는 한 Producer 는 실패한 전송을 다시 보냅니다. 멥등성이 없으면 그 재시도가 곧 중복입니다.

순서와 재시도

- `max.in.flight.requests.per.connection` : 응답을 받지 않고 동시에 보낼 수 있는 요청 수입니다
- **역등성이 꺼져 있으면**: 1보다 크면 순서가 뒤집힐 수 있습니다. 앞 요청이 재시도되는 동안 뒤 요청이 먼저 저장됩니다
- **역등성이 켜져 있으면**: 5까지 순서가 유지됩니다. Broker 가 시퀀스 번호로 다시 맞춥니다
- `retries` : 기본이 사실상 무제한입니다. `delivery.timeout.ms` 가 끝을 정합니다

순서 보장은 Partition 안에서, 그리고 Producer 설정이 맞을 때 성립합니다.

키로 같은 Partition 에 보내도 이 설정이 어긋나면 순서가 뒤집힙니다.

compression.type

값	압축률	특징
<code>none</code>	없음	CPU 를 쓰지 않습니다
<code>gzip</code>	높음	CPU 를 많이 씁니다
<code>snappy</code>	낮음	빠릅니다
<code>lz4</code>	중간	빠르고 균형이 좋습니다
<code>zstd</code>	높음	압축률과 속도가 모두 좋습니다. 최근의 기본 선택입니다

- **Producer 가 압축합니다:** 배치 단위로 압축합니다
- **Broker 는 그대로 저장합니다:** 풀지 않습니다. 디스크와 네트워크를 함께 아킵니다
- **Consumer 가 풋니다:** 양쪽 클라이언트가 CPU 를 씁니다

압축은 네트워크·디스크와 CPU 를 맞바꿉니다. Broker CPU 는 거의 쓰지 않습니다. 클라이언트 쪽 여유를 보고 고릅니다.

06

Consumer Group

그룹과 할당

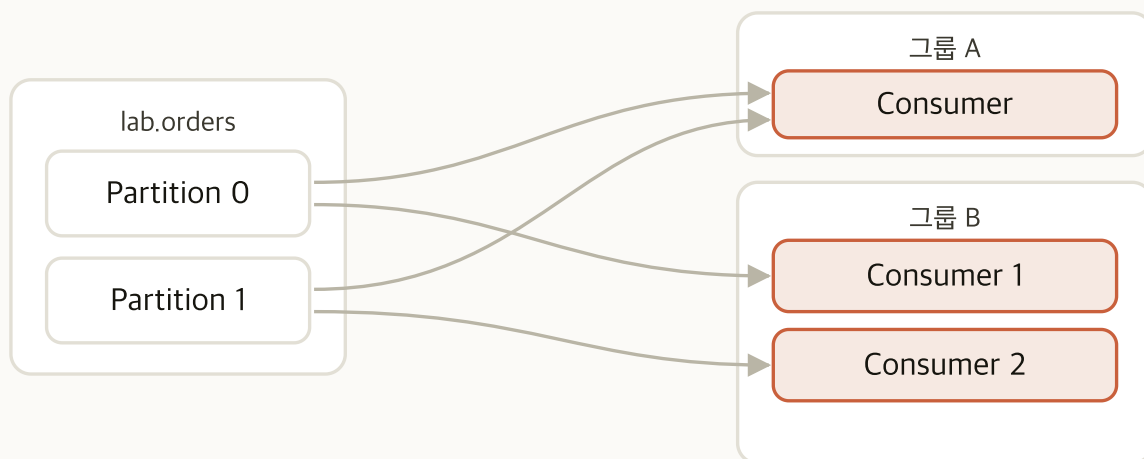
Rebalance

할당 전략

timeout 세 가지

static membership

Consumer Group



- **그룹 안에서는 나눠 갖습니다:** Partition 하나는 Consumer 하나입니다
- **그룹끼리는 독립입니다:** 같은 메시지를 각자 읽습니다
- **그룹끼리 할당이 얹히지 않습니다:** 한 그룹의 Consumer 가 늘거나 줄어도 다른 그룹의 배분은 그대로입니다

group.id 가 같으면 나눠 읽고, 다르면 각자 다 읽습니다.

같은 데이터를 여러 용도로 쓸 때 그룹을 나눕니다.

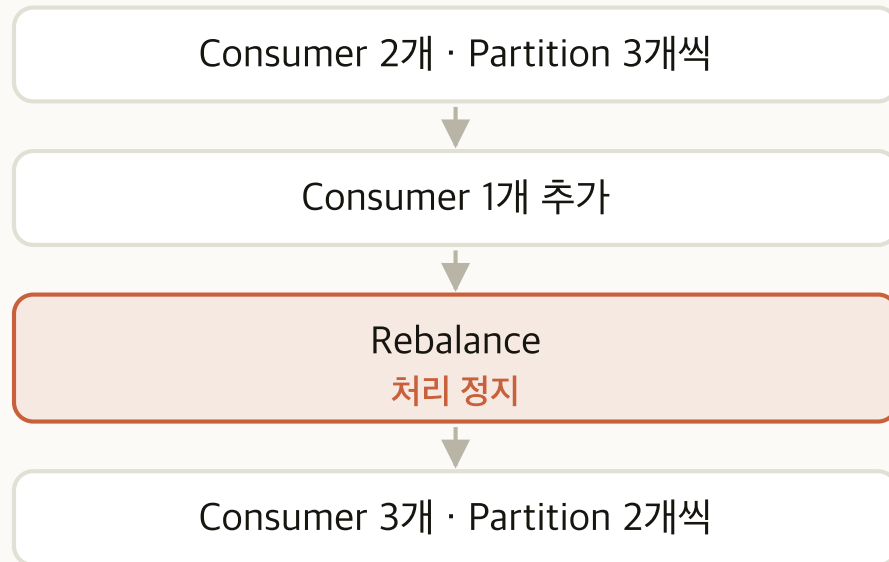
Consumer 수와 Partition 할당

Partition 6개일 때 Consumer 수	할당	결과
1	한 Consumer 가 6개	병렬성 없음
2	3개씩	고르게 나뉨
3	2개씩	고르게 나뉨
4	2·2·1·1	고르지 않음
6	1개씩	최대 병렬성
7	여섯이 1개씩, 하나는 0개	할당 없는 Consumer 발생

Consumer 를 Partition 수보다 많이 띄우는 것은 낭비입니다. 할당을 받지 못한 Consumer 도 그룹 멤버라 Rebalance 에 참여하고 하트비트를 보냅니다.

Rebalance

Rebalance 는 그룹 안의 **Consumer** 들에게 **Partition** 을 다시 나눠 주는 것입니다. 멤버가 바뀌면 누가 어느 **Partition** 을 맡을지 처음부터 다시 정합니다.



- 일어나는 조건: 멤버 추가·이탈, 구독 Topic 변경, **Partition 수 변경**
- 그동안 처리가 멈춥니다: 할당이 정해질 때까지 아무도 읽지 않습니다
- 잦으면 처리가 밀립니다: 멈춘 시간만큼 뒤로 밀립니다

Rebalance 는 정상 동작이지만 잦으면 장애처럼 보입니다.

Consumer 는 살아 있는데 처리가 진행되지 않는 경우가 여기에 해당합니다.

할당 전략

누가 어느 Partition 을 맡을지 정하는 규칙입니다. 4.x 에는 프로토콜이 두 가지 있고, 기본은 classic 입니다.

전략	나누는 방식	Rebalance 중
Range	Topic 마다 Partition 을 순서대로 잘라 나눕니다	전원 정지 (eager)
RoundRobin	전체 Partition 을 돌아가며 나눕니다	전원 정지 (eager)
Sticky	기존 할당을 최대한 유지합니다	전원 정지 (eager)
CooperativeSticky	기존 할당을 유지하며 필요한 것만 옮깁니다	옮기지 않는 Consumer 는 계속 처리합니다 (cooperative)

기본값은 **RangeAssignor, CooperativeStickyAssignor** 이고 앞에서부터 모든 멤버가 지원하는 첫 전략이 쓰입니다. 값을 바꾸지 않으면 Range 로 동작합니다.

timeout 세 가지

설정	무엇인가	어긋나면
<code>heartbeat.interval.ms</code>	하트비트를 보내는 주기	너무 길면 정지를 늦게 알아챱니다. <code>session.timeout.ms</code> 의 3분의 1이 권장입니다
<code>session.timeout.ms</code>	하트비트를 이 시간 동안 못 받으면 정지로 판정하는 한 계	그룹에서 빼고 Rebalance 를 돌립니다
<code>max.poll.interval.ms</code>	<code>poll()</code> 사이에 허용되는 최대 간격	처리가 길면 살아 있어도 그룹에서 제외됩니다

셋 중 실제로 장애로 이어지는 것은 `max.poll.interval.ms` 입니다.

하트비트는 별도 스레드가 보내므로 메시지 처리가 길어져도 계속 전달됩니다. 그런데 메시지 처리가 길어 `poll()` 을 못 부르면 그룹에서 제외됩니다.

static membership

Consumer 애플리케이션을 배포하는 과정에서 불필요한 Rebalance 가 일어나지 않게 하는 설정입니다. 재시작은 짧게 이탈했다가 복귀하는 것인데, 기본 동작은 그때마다 새 멤버로 보고 Partition 을 다시 나눕니다.

- `group.instance.id` : Consumer 에 고정 ID 를 붙입니다
- 무엇이 달라지는가: 재시작해도 같은 멤버로 인식해 Rebalance 를 건너뛵니다
- 주의: `session.timeout.ms` 안에 돌아와야 합니다. 넘으면 결국 Rebalance 가 일어납니다
- 기본으로 켜는 설정이 아닙니다: 고정 ID 를 부여하고 관리해야 하고, 정지한 멤버를 늦게 알아채는 대가가 따릅니다

Consumer 수가 많고 배포가 잦을 때만 효과가 있습니다.
그 조건이 아니면 켜지 않습니다.

Consumer Group 구성

- 1 터미널 둘을 열고 각각 `kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --formatter-property print.partition=true --command-property client.id=consumer-1`. 둘째 터미널은 `client.id=consumer-2` 로
- 2 세 번째 터미널에서 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg` 로 할당 확인
- 3 세 번째 터미널에서 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --reader-property parse.key=true --reader-property key.separator=:` 로 키를 붙여 여러 건 produce
- 4 Consumer 두 터미널의 출력에서 Partition 번호 확인
- 5 네 번째 터미널에서 세 번째 Consumer 를 `client.id=consumer-3` 으로 띄우고, 세 번째 터미널에서 `--describe` 로 할당 재확인

완료 판정

Consumer 2개일 때 Partition 이 3개씩, 3개일 때 2개씩 나뉘면 완료입니다.

--describe --group 출력

```
GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
lab-cg     lab.orders  0          2               2               0    consumer-1-...
lab-cg     lab.orders  1          3               3               0    consumer-1-...
lab-cg     lab.orders  2          1               1               0    consumer-1-...
lab-cg     lab.orders  3          2               2               0    consumer-2-...
... Partition 4·5 가 consumer-2 로 이어집니다
```

열	뜻	판정
PARTITION	그 그룹이 맡고 있는 Partition 번호	여섯이 모두 나와야 합니다
CONSUMER-ID	그 Partition 을 맡은 Consumer	두 Consumer 가 세 개씩 맡았습니다

Partition 수를 넘는 Consumer 의 할당

- 1 다섯 번째 터미널에서 `for i in 4 5 6 7; do kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --command-property client.id=consumer-$i > /tmp/consumer-$i.out 2>&1 & done` 로 넷을 더 띄움. p.54 의 셋과 합쳐 일곱
- 2 Rebalance 가 끝날 때까지 기다림
- 3 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg --members` 로 멤버별 할당 확인
- 4 Partition 을 하나도 못 받은 멤버가 있는지 확인
- 5 `pkill -f "client.id=consumer-[34567]"` 로 셋째부터 일곱째까지 정지시켜 2개로 되돌림

완료 판정

`--members` 출력에서 `#PARTITIONS` 가 0인 멤버가 하나 보이면 완료입니다.

--describe --members 출력

```
GROUP      CONSUMER-ID      HOST      CLIENT-ID      #PARTITIONS
lab-cg     consumer-1-...    /10.0.1.4  consumer-1     1
... consumer-2 부터 consumer-6 까지 각각 1
lab-cg     consumer-7-...    /10.0.1.4  consumer-7     0
```

- **#PARTITIONS** 가 0인 멤버: 아무것도 할당받지 못했습니다
- 그래도 그룹 멤버입니다: 하트비트를 보내고 Rebalance 에 참여합니다
- Consumer 를 늘려도 처리량이 오르지 않습니다: 상한은 Partition 수입니다

처리가 밀릴 때 Consumer 를 늘리는 것이 항상 답은 아닙니다.

Partition 수에 이미 도달했으면 Partition 을 늘려야 하고, 그것은 되돌릴 수 없는 결정입니다.

07

Offset

Offset

Offset 저장

커밋 시점

전달 보장

시작 위치

Offset 조정

Offset



- **정의:** Partition 안에서 메시지에 붙는 단조 증가 번호입니다
- **매겨지는 범위:** **Partition 안에서만**입니다. Topic 전체를 아우르는 번호가 없습니다
- **누가 기억하는가:** Broker 가 아니라 Consumer Group 이 기억합니다
- **되돌릴 수 있는가:** 있습니다. 이 챕터 실습에서 직접 되돌립니다

Offset 은 Partition 마다 따로 매겨집니다.

커밋도 되감기도 Partition 마다 따로 이뤄지고, 순서 보장도 Partition 안에서만 성립합니다.

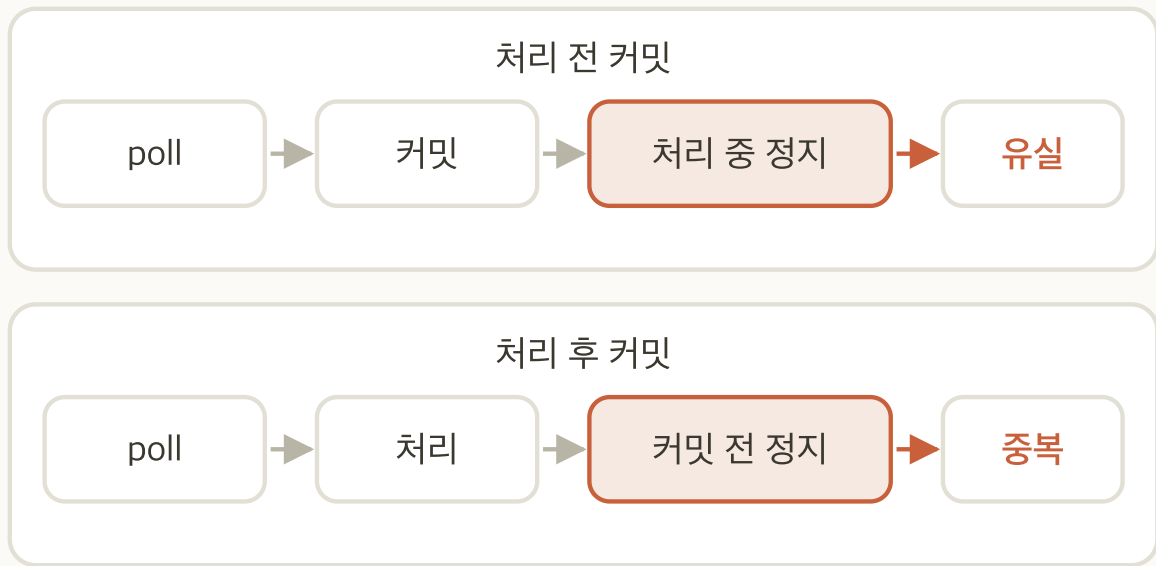
__consumer_offsets

- **저장 위치**: Kafka 의 내부 Topic 입니다. 파일이나 외부 저장소가 아닙니다
- **키**: 그룹 ID, Topic, Partition 의 조합입니다
- **값**: 그 그룹이 그 Partition 을 어디까지 처리했는지입니다
- **cleanup.policy=compact** : 키별 최신 값만 남깁니다. 그래서 무한히 커지지 않습니다

Offset 도 Kafka Topic 이라 운영 대상입니다.

이 Topic 이 문제를 겪으면 모든 Consumer Group 이 위치를 잃습니다. 기본 RF 는 3입니다.

커밋 시점과 전달 보장



- **처리 전 커밋:** 처리 중에 정지하면 그 메시지를 잃습니다
- **처리 후 커밋:** 커밋 전에 정지하면 다시 처리합니다
- **auto commit:** `auto.commit.interval.ms` 마다 자동으로 커밋합니다. 처리가 끝났는지와 무관합니다

유실과 중복을 둘 다 피하는 기본값이 없습니다.

무엇을 감수할지 고르는 것이 운영 판단입니다. 대부분 중복을 고르고 처리를 멍등하게 만듭니다.

전달 보장 3종

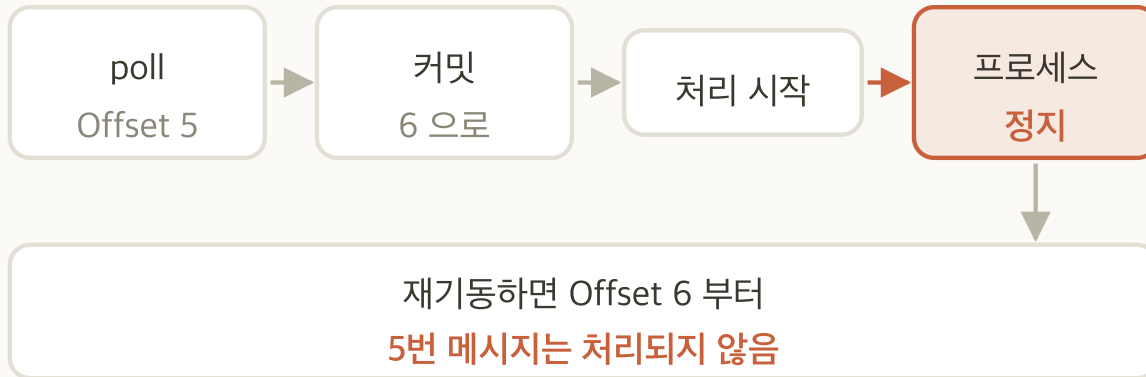
보장	어떻게 얻는가	대가
at-most-once	처리 전에 커밋	유실이 생길 수 있습니다
at-least-once	처리 후에 커밋	중복이 생길 수 있습니다. 가장 흔한 선택입니다
exactly-once	트랜잭션과 멍등 Producer	설정과 코드가 늘고 처리량이 떨어집니다

exactly-once 는 Kafka 안에서 닫힌 처리에만 성립합니다.

읽어서 외부 DB 에 쓰는 경우는 그쪽까지 함께 트랜잭션에 넣어야 합니다. 이 과정의 범위 밖입니다.

at-most-once

처리하기 전에 커밋합니다. 한 번 커밋한 메시지는 다시 받지 않으므로 중복이 없고, 대신 처리 중에 정지하면 그 메시지가 사라집니다.



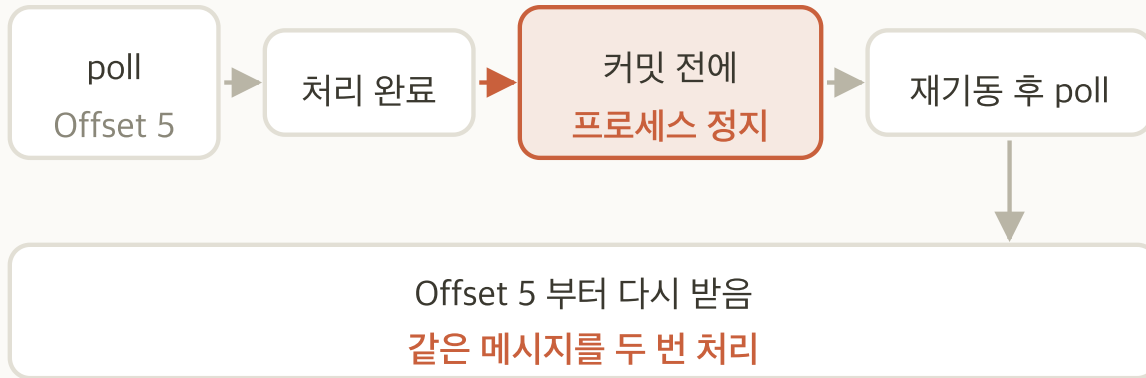
- 어떻게 얻는가: 처리 전에 직접 커밋합니다.
`enable.auto.commit` 은 직전 `poll()` 이 돌려준 위치를 커밋하므로 이 보장이 성립하지 않습니다
- 중복: 없습니다
- 유실: 있습니다. 커밋과 처리 완료 사이에 정지하면 그 구간이 사라집니다

다시 받지 않아도 되는 데이터에만 씁니다.

지표 수집처럼 한두 건이 빠져도 결과가 크게 달라지지 않는 경우입니다.

at-least-once

처리한 뒤에 커밋합니다. 유실이 없는 대신 커밋 전에 정지하면 이미 처리한 메시지를 다시 받습니다.



- **어떻게 얻는가:** `enable.auto.commit=false` 로 두고 처리가 끝난 뒤 직접 커밋합니다
- **유실:** 없습니다. **중복:** 있습니다
- **멥등한 처리:** 같은 메시지를 두 번 처리해도 결과가 같게 만드는 것입니다. 메시지 ID 로 처리 이력을 확인하거나, 더하기 대신 뺄어쓰기로 씁니다

실무의 기본값입니다.

유실을 막고 중복은 처리 쪽에서 흡수합니다.

exactly-once

처리·쓰기·Offset 커밋을 하나의 트랜잭션으로 묶습니다. 중간에 정지하면 그 트랜잭션 전체가 없던 일이 되므로 유실도 중복도 없습니다.



- 어떻게 얻는가: 맥등 Producer 와 트랜잭션 API 를 함께 씁니다. Offset 커밋도 그 트랜잭션에 넣습니다
- 대가: 설정과 코드가 늘고 처리량이 떨어집니다
- 성립 범위: Kafka 안에서 읽어 Kafka 안에 쓰는 처리입니다

읽어서 외부 DB 에 쓰는 경우에는 이것만으로 성립하지 않습니다.

그쪽 쓰기까지 함께 트랜잭션에 넣거나 DB 쪽에서 중복을 걸러야 합니다. 이 과정의 범위 밖입니다.

auto.offset.reset

값	커밋된 Offset 이 없을 때	언제 쓰는가
latest (기본)	지금부터 들어오는 것만 읽습니다	과거가 필요 없는 실시간 처리
earliest	남아 있는 가장 오래된 것부터 읽습니다	처음부터 다시 처리해야 하는 경우
none	예외를 냅니다	Offset 이 없는 상황 자체를 오류로 다뤄야 할 때

새 그룹으로 접속했는데 아무것도 읽히지 않으면 이 값을 봅니다.
기본이 **latest** 라 이미 쌓인 메시지를 건너웁니다.

Consumer Group Offset 조정

`kafka-consumer-groups.sh --reset-offsets` 는 특정 Consumer Group 의 처리 위치를 사람이 직접 옮기는 명령입니다. Topic 의 메시지는 변경하지 않고 그 그룹의 위치만 바꿉니다.

옵션	어디로 옮기는가	쓰는 곳
<code>--to-earliest</code>	남아 있는 가장 오래된 위치	전체 재처리
<code>--to-latest</code>	마지막 위치	밀린 것을 버리고 지금부터
<code>--to-offset <N> · --shift-by <N></code>	지정한 Offset · 현재 위치에서 앞뒤로	특정 지점부터 다시. 조금만 되감을 때
<code>--to-datetime <시각></code>	그 시각 이후 첫 메시지	장애 발생 시각부터 재처리

`--execute` 는 되돌릴 수 없습니다. `--dry-run` 이 기본이며, 그룹이 활성화면 거부되므로 Consumer 를 먼저 정지시킵니다.

Offset 되감기

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic __consumer_offsets | head -1` 로 정책이 `compact` 인 것 확인
- 2 Consumer 를 전부 정지한 뒤 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg` 로 현재 `CURRENT-OFFSET` 을 기록
- 3 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --group lab-cg --topic lab.orders --reset-offsets --to-earliest --dry-run`
- 4 출력의 `NEW-OFFSET` 열 확인. **아직 반영되지 않았습니다**
- 5 같은 명령에서 `--dry-run` 을 `--execute` 로 바꿔 실행
- 6 `--describe` 로 `CURRENT-OFFSET` 이 0이 된 것 확인 후 Consumer 재기동

`--execute` 는 되돌릴 수 없습니다.

`--describe` 출력의 `CURRENT-OFFSET` 을 먼저 적어 두지 않으면 원래 위치로 돌아올 수 없습니다.

--dry-run 과 --execute

```
GROUP    TOPIC      PARTITION  NEW-OFFSET
lab-cg    lab.orders    0          0
lab-cg    lab.orders    1          0
lab-cg    lab.orders    2          0
... Partition 3·4·5 도 같은 값입니다
```

- **--dry-run** 과 **--execute** 의 출력이 같습니다: 실제로 반영됐는지는 **--describe** 로 확인합니다
- **NEW-OFFSET** 이 0: **--to-earliest** 이고 아직 아무것도 삭제되지 않았습니다

--to-earliest 는 0이 아니라 「남아 있는 가장 오래된 것」입니다.

지금은 아무것도 지워지지 않아 0입니다. 오래된 부분이 지워진 뒤에는 0이 아닙니다.

08

로그 기반 메시징과 저장 구조

로그

segment

활성 segment

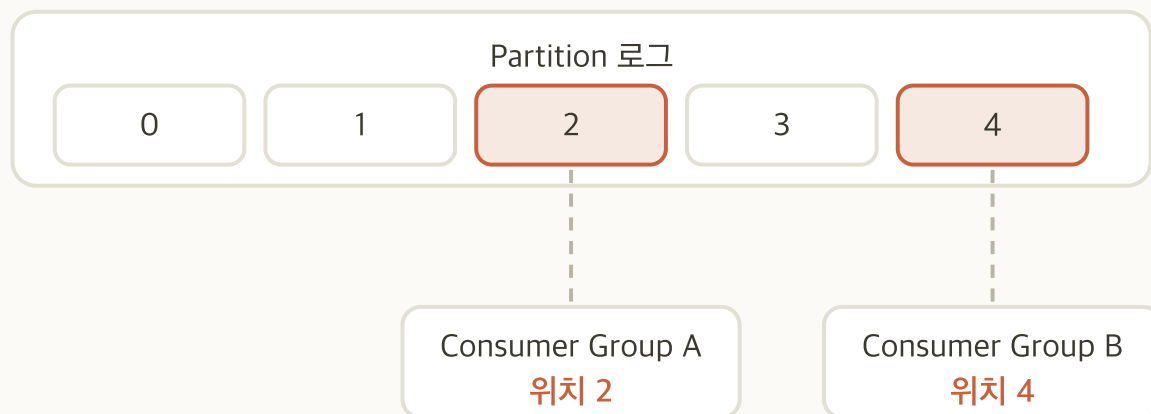
OS 캐시

zero-copy

순차 처리량

로그 기반 메시징

Kafka 는 메시지를 **끝에만 덧붙이는 로그 파일**에 씁니다. 읽는 쪽은 자기가 어디까지 처리했는지를 스스로 기억합니다.

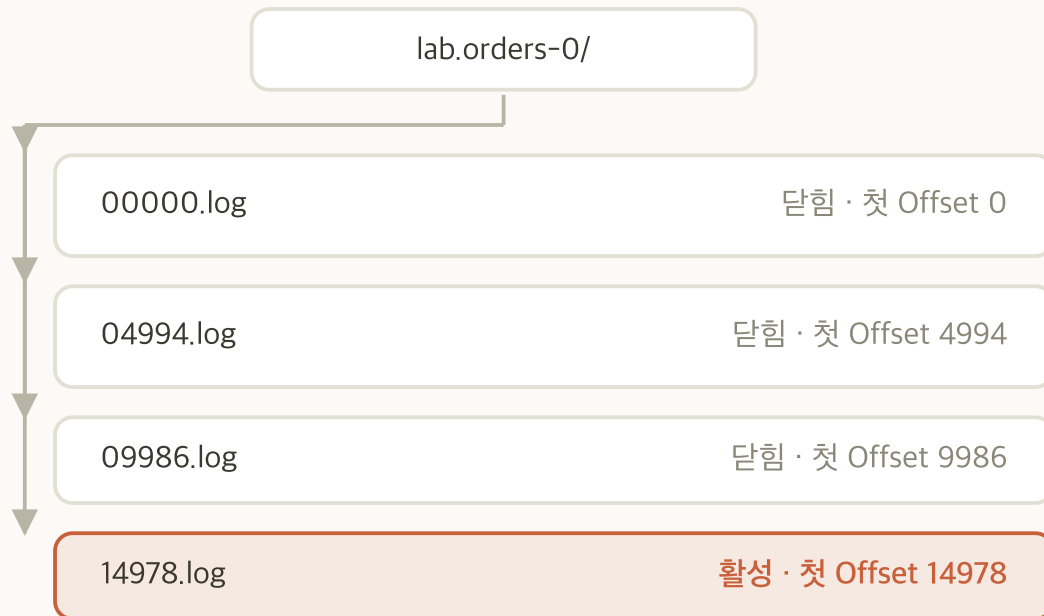


- **읽어도 사라지지 않습니다:** 위치만 옮깁니다
- **여러 그룹이 독립적으로 읽습니다:** 서로의 위치에 영향을 주지 않습니다
- **재처리가 가능합니다:** 위치를 앞으로 되돌리면 같은 메시지를 다시 읽습니다

큐와 가장 크게 갈리는 지점입니다.

큐는 꺼내면 없어집니다. 로그는 남아 있고 언제까지 남길지는 Retention 이 정합니다.

segment 파일



- **롤링:** `log.segment.bytes` 를 넘거나 `log.roll.ms` 가 지나면 새 파일을 엽니다. Topic 단위로 걸 때는 앞의 `log.` 를 뺀 `segment.bytes` · `segment.ms` 를 씁니다
- **닫힘:** 이전 파일에는 더 쓰지 않습니다. 내용이 고정되고 **마지막 메시지 시각이 확정됩니다**
- **활성 segment:** 지금 쓰고 있는 파일 하나입니다. 쓰기가 이어지는 동안에는 시각이 갱신되어 **삭제 판정에 들어가지 않습니다**

메시지 하나만 지울 수는 없습니다.

중간을 지우면 그 뒤 Offset 이 전부 밀리는데, Offset 은 Consumer 가 기억하는 값이라 바뀌면 안 됩니다. 그래서 삭제 단위가 파일입니다.

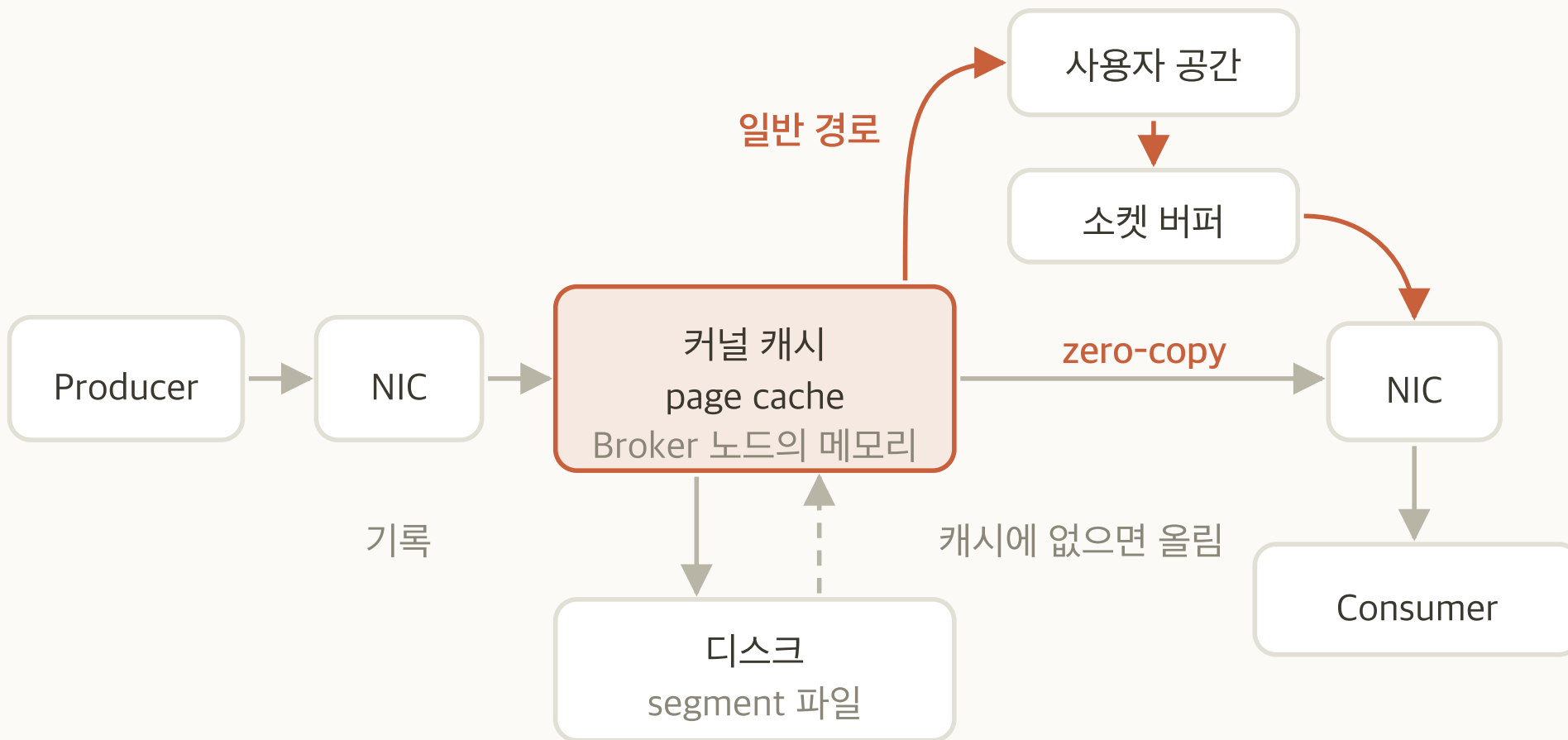
디스크가 병목이 되지 않는 이유

기법	무엇을 하는가	효과
순차 IO	끝에만 덧붙이고 끝에서부터 읽습니다	디스크 탐색이 거의 없습니다
OS 캐시	방금 쓴 데이터가 캐시에 남아 있습니다	최근 데이터를 읽는 Consumer 는 디스크에 닿지 않습니다
zero-copy (<code>sendfile</code>)	커널이 파일에서 소켓으로 바로 보냅니다	메시지가 사용자 공간과 JVM 힙을 거치지 않습니다

셋 다 「메모리에 올려 두기」가 아닙니다.

Kafka 는 메시지를 힙에 캐싱하지 않습니다. 힙은 요청 버퍼와 메타데이터에 쓰이고 메시지는 OS 캐시가 맡습니다.

메시지의 이동 경로



들어오는 길과 나가는 길이 커널 캐시에서 만납니다. 방금 들어온 메시지를 곧바로 읽어 가면 디스크까지 내려가지 않습니다.

두 복사 경로의 대비

커널 캐시(page cache)는 커널이 디스크 내용을 들고 있는 Broker 노드의 메모리이고, NIC 는 네트워크 카드입니다. 파일을 네트워크로 보내려면 이 둘 사이를 지나야 합니다.

항목	일반 경로	zero-copy
거치는 곳	커널 캐시 → 사용자 공간 버퍼 → 소켓 버퍼 → NIC	커널 캐시 → NIC
복사 횟수	4회	2회
시스템 호출	2회 (read 와 write)	1회 (sendfile)
CPU 가 하는 일	버퍼 사이를 두 번 복사	거의 없음

디스크가 빨라서가 아니라 거치는 단계가 적어서입니다.

Kafka 는 메시지를 가공하지 않고 그대로 내보내므로 사용자 공간으로 올릴 이유가 없습니다. Broker CPU 가 메시지를 가공하지 않는 것이 이 구조의 전제입니다.

디스크 성능 지표와 Kafka 의 병목

지표	무엇을 재는가	Kafka 에서
IOPS	초당 처리하는 IO 요청 수	Kafka 는 segment 끝에만 이어 씁니다. 흩어진 위치에 접근하지 않으므로 큰 덩어리 하나가 요청 하나가 되고, 같은 바이트를 옮기는 데 필요한 요청 수가 적습니다
처리량 (MB/s)	초당 옮기는 바이트	여기가 상한입니다. 요청 수가 아니라 옮기는 양이 먼저 한계에 닿습니다
지연	요청 하나의 응답 시간	최근 데이터는 OS 캐시에서 응답하므로 디스크까지 내려가지 않습니다. 디스크 지연을 낮춰도 효과가 크지 않고, 캐시를 벗어난 읽기에서만 그대로 드러납니다

디스크를 고를 때 볼 것은 순차 처리량입니다.

랜덤 IOPS 를 올려도 효과가 크지 않습니다. 흩어진 위치를 읽는 MongoDB 와 정반대입니다.

뒤쳐진 Consumer 가 미치는 영향

OS 캐시는 Broker 노드의 메모리에 있습니다. 디스크에서 읽은 파일 내용을 커널이 메모리에 들고 있다가 다음 요청에 그대로 내주는 구조이고, 자리가 모자라면 오래 쓰이지 않은 것부터 밀어냅니다.

- ① 최근 데이터는 이미 메모리에 있습니다: 방금 쓴 것이라 캐시에 남아 있어 Consumer 가 디스크에 닿지 않습니다
- ② 오래된 Offset 을 읽으면 디스크에서 올려야 합니다: 캐시에 없으므로 그만큼 느립니다
- ③ 그 과정에서 최근 데이터가 캐시에서 밀려납니다: 커널이 자리를 만들려고 오래 쓰이지 않은 것부터 버립니다
- ④ 최근 데이터를 읽던 Consumer 까지 디스크로 내려가게 됩니다: 그 Consumer 는 달라진 것이 없는데 읽기만 갑자기 느려집니다

한 Consumer 가 뒤처지면 그 Consumer 만의 문제로 끝나지 않습니다.

뒤쳐진 Consumer 하나가 오래된 구간을 훑는 동안 Broker 전체의 읽기가 디스크로 내려갑니다.

09

KRaft 와 클러스터 메타데이터

메타데이터

Raft

controller

quorum

combined 모드

quorum 상태

클러스터 메타데이터

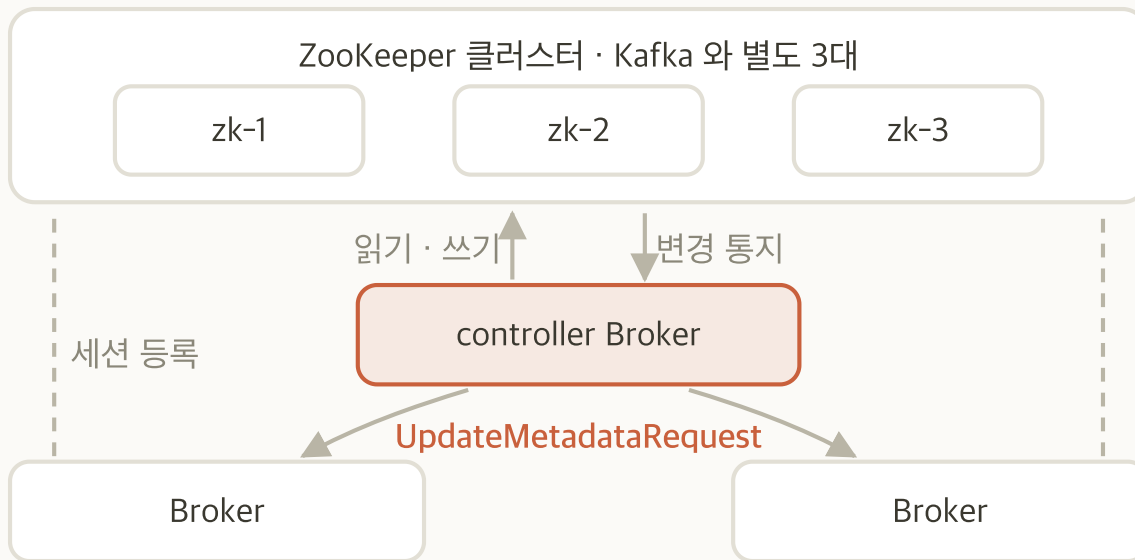
- **정의:** Topic 목록, Partition 배치, 각 Partition 의 leader 와 복제본, Broker 목록입니다
- **왜 합의가 필요한가:** Broker 마다 다르게 알고 있으면 클라이언트가 잘못된 Broker 로 요청을 보냅니다
- **어디에 저장되는가:** `__cluster_metadata` 라는 내부 Topic 입니다. **메타데이터도 로그입니다**
- **누가 쓰는가:** controller 중 leader 하나만 씁니다. 나머지는 그 로그를 따라 읽습니다

메타데이터가 멈추면 데이터는 흘러도 구성이 바뀌지 않습니다.

Topic 생성, Partition 재배포, leader 교체가 전부 멈춥니다.

ZooKeeper 의 역할

4.0 이전에는 Kafka 옆에 ZooKeeper 클러스터가 따로 떠 있었습니다. 무엇을 맡았는지 알아야 KRaft 가 무엇을 대신하는지 보입니다.

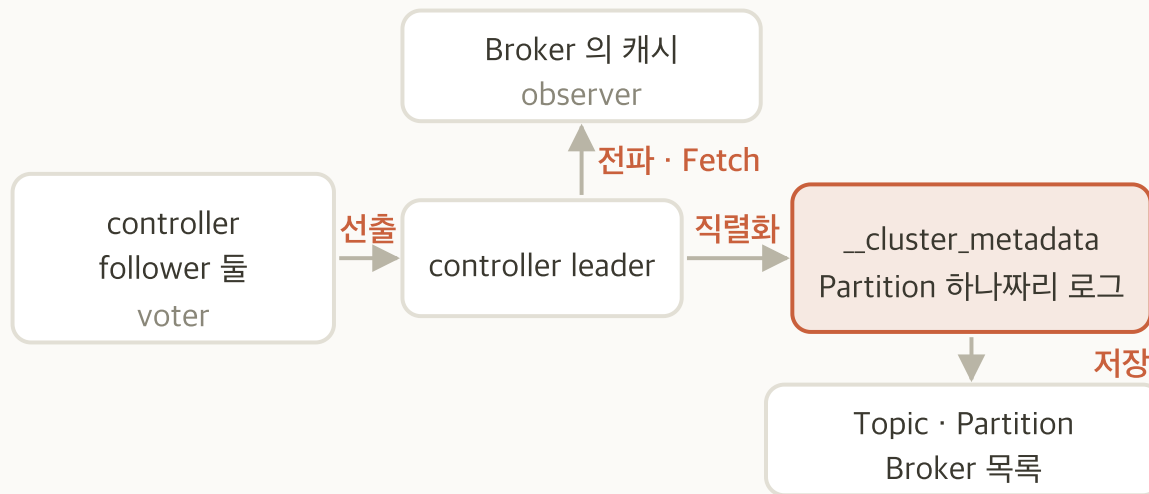


- **메타데이터 저장:** Topic·Partition·설정을 ZooKeeper 에 두었습니다
- **controller 선출:** 어느 Broker 가 controller 가 될지 ZooKeeper 가 정했습니다
- **Broker 등록과 감시:** 세션이 끊기면 등록이 사라지고 ZooKeeper 가 controller 에 알렸습니다
- **Broker 는 ZooKeeper 를 읽지 않았습니다:** controller 가 Broker 마다 따로 보내 줬습니다

운영 대상이 둘이었습니다. 버전·백업·모니터링·장애 대응이 각각 두 벌이었습니다.

KRaft 의 정의와 역할

KRaft 는 Kafka Raft 입니다. Raft 는 여러 노드가 하나의 값에 합의하는 알고리즘이고, KRaft 는 그것을 Kafka 자신의 로그 위에 구현한 것입니다.



- **선출**: controller 들이 투표해 leader 하나를 뽑습니다
- **직렬화**: leader 만 씁니다. Offset 이 변경의 순서를 정합니다
- **저장**: 로그의 이름이 `___cluster_metadata` 입니다. Topic 목록, Partition 배치, Broker 목록이 담깁니다
- **전파**: Broker 는 **leader 에게서** 로그를 `Fetch` 로 당겨 읽어 자기 캐시를 채웁니다

controller follower 는 가용성을 위해 있고, 실제로 Kafka 를 돌리는 데 쓰이는 것은 Broker 의 메타데이터 캐시입니다.

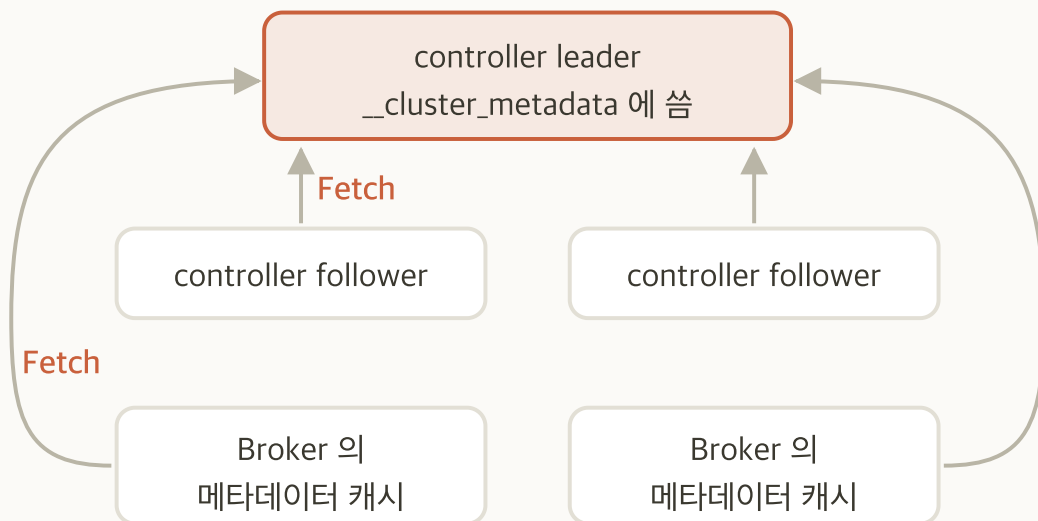
follower 는 투표하고 과반을 세는 voter 입니다. leader 가 정지하면 그중 하나가 이어받습니다. Broker 는 투표하지 않는 observer 로, leader 에게서 같은 로그를 당겨 읽어 어느 Partition 이 어디 있는지를 압니다.

ZooKeeper 에서 KRaft 로

항목	ZooKeeper 시절	KRaft
메타데이터 저장소	별도 클러스터	Kafka 자신의 로그
운영 대상	Kafka 와 ZooKeeper 둘	Kafka 하나
메타데이터 전파	controller 가 Broker 에 개별 통지	Broker 가 로그를 따라 읽음
대규모에서	Partition 수가 늘면 통지가 병목	로그 복제라 같은 방식으로 확장

4.0에서 ZooKeeper 가 제거됐습니다.
 이 과정에서는 KRaft 구성만 다룹니다. 4.0 이전 자료를 볼 때 구성이 다른 것은 이 때문입니다.

KRaft 의 구조

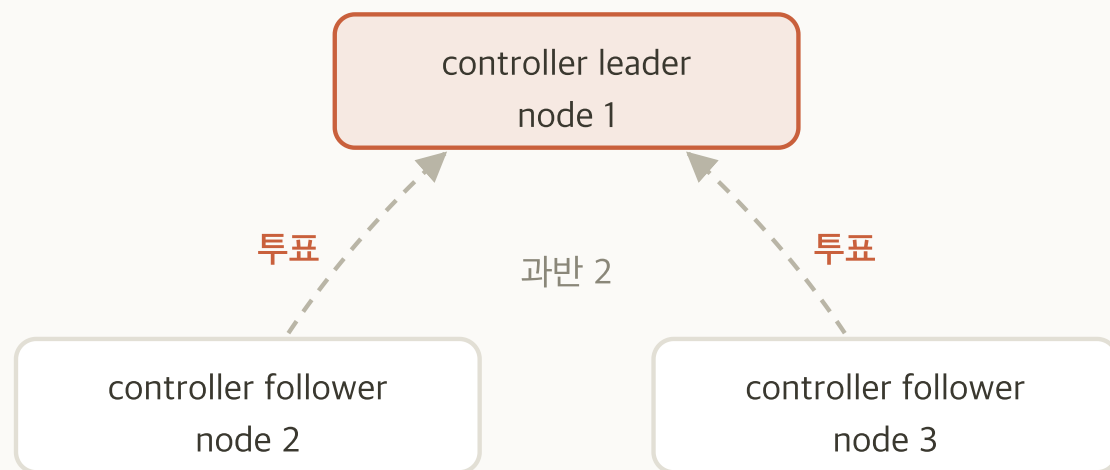


- 쓰는 곳은 하나입니다: controller leader 만
__cluster_metadata 에 씁니다
- 나머지는 당겨 읽습니다: follower 도 Broker 도 Fetch 로 같은 로그를 순서대로 읽습니다
- 통지가 아니라 복제입니다: ZooKeeper 시절의 개별 통지와 갈리는 지점입니다
- Broker 는 투표하지 않습니다: 메타데이터를 읽기만 하는 observer 입니다

메타데이터도 Partition 하나짜리 로그입니다.

Offset 이 있고 순서가 있으며 따라잡기가 있습니다. Broker 가 재기동하면 마지막으로 읽은 Offset 부터 이어 읽습니다.

controller quorum



- 과반이 필요합니다: 3대 구성에서 2입니다
- leader 는 하나입니다: `ActiveControllerCount` 가 정확히 1이어야 합니다
- 1대가 정지해도 견딥니다: 남은 2대가 과반이라 메타데이터 변경이 계속됩니다
- 2대가 정지하면 멈춥니다: 과반을 못 채워 새 leader 를 뽑지 못합니다

`ActiveControllerCount` 가 0이면 메타데이터 변경이 멈춥니다.

데이터는 계속 흐르지만 Topic 생성·Partition 재배포·leader 교체가 전부 멈춥니다.

combined 모드와 isolated 모드

항목	combined (이 실습)	isolated
프로세스	Broker 와 controller 가 한 프로세스	서로 다른 프로세스
노드 수	Broker 3대면 3대	Broker 3대 + controller 3대 = 6대
장애 영향	Broker 가 정지하면 controller 도 함께 정지합니다	서로 독립적입니다
롤링과 증설	controller 만 따로 롤링·증설할 수 없습니다	따로 롤링·증설할 수 있습니다
권장	개발과 소규모에 한함	운영 권장

Apache Kafka 는 운영 환경에서 combined 모드를 권장하지 않습니다.

controller 가 나머지와 격리되지 않고, controller 를 Broker 와 따로 롤링하거나 늘릴 수 없기 때문입니다. 이 실습이 combined 인 것은 분리하면 controller 전용 노드 세 대가 더 필요해서입니다.

내부 Topic

Topic	내용
<code>__cluster_metadata</code>	클러스터 메타데이터. KRaft 의 로그
<code>__consumer_offsets</code>	Consumer Group 의 Offset
<code>__transaction_state</code>	트랜잭션 상태. 이 과정의 범위 밖입니다

- `cleanup.policy=compact` : 키별 최신 값만 남깁니다
- 운영 대상입니다: 내부 Topic 도 디스크를 쓰고 복제됩니다

Kafka 는 자기 상태도 자기 Topic 에 담습니다.

메타데이터도 Offset 도 별도 저장소가 아니라 로그입니다. 그래서 Offset 이 있고 순서가 있으며 복제됩니다.

클러스터 상태 확인

- 1 `kafka-metadata-quorum.sh --bootstrap-server 10.0.1.5:9092 describe --status` 로 quorum 상태 확인
- 2 출력에서 `LeaderId` 와 `CurrentVoters` 기록
- 3 `kafka-metadata-quorum.sh --bootstrap-server 10.0.1.5:9092 describe --replication` 으로 세 노드의 복제 지연 확인
- 4 `kafka-broker-api-versions.sh --bootstrap-server 10.0.1.5:9092 | grep "id:"` 로 Broker 3대 확인

완료 판정

quorum leader 가 하나이고 `CurrentVoters` 에 세 노드가 모두 있으면 완료입니다.

describe --status 출력

```
ClusterId:      A4lrm9UeR0qyJfRZk0xjDw   LeaderId:  1   LeaderEpoch:  1
HighWatermark:  2614   MaxFollowerLag:  0   MaxFollowerLagTimeMs:  0
CurrentVoters:  [{"id": 1, "endpoints": ["CONTROLLER://10.0.1.5:9093"]}, ...]
CurrentObservers:  []
```

필드	뜻	판정
LeaderId	지금 controller leader 의 <code>node.id</code>	값이 있어야 합니다. 비어 있으면 leader 가 없는 상태입니다
LeaderEpoch	leader 가 바뀐 횟수	오르면 controller 가 교체된 것입니다
CurrentVoters	투표권이 있는 노드	세 노드가 모두 있어야 합니다
CurrentObservers	투표권 없이 읽기만 하는 노드	비어 있습니다. combined 모드에서는 같은 노드가 이미 voter 입니다

10

복제와 ISR

복제본 역할

ISR

replication.factor

응답 조건

unclean 선출

High Watermark

leader 와 follower



- 읽기와 쓰기는 leader 가 받습니다: follower 는 복제만 합니다
- follower 를 늘려도 처리량은 오르지 않습니다: 내구성만 오릅니다
- ISR (In-Sync Replica) 은 leader 를 따라잡은 복제본의 집합입니다: leader 자신도 포함됩니다

Partition 마다 leader 가 따로 있습니다.

`lab.orders` 는 Partition 이 6개이므로 leader 도 6개이고, 세 Broker 에 나뉘어 있습니다.

replication.factor

- **정의:** Partition 하나를 몇 개의 Broker에 둘지 정하는 값입니다. leader를 포함한 수입니다
- **상한:** Broker 수입니다. 복제본은 서로 다른 Broker에 놓입니다
- **표준값:** 3입니다. 1대가 정지해도 복제본이 둘 남습니다
- **바꿀 수 있는가:** 만든 뒤에는 Partition 재배치로만 바꿉니다. `--alter`로는 되지 않습니다

RF=3 만으로는 내구성이 정해지지 않습니다.

세 곳에 둘 자리를 만든 것일 뿐입니다. 실제로 몇 곳이 받고 나서 응답할지는 `acks`와 `min.insync.replicas`가 정합니다.

ISR

항목	내용	운영상 의미
정의	leader 를 충분히 따라잡은 복제본의 집합	leader 자신도 포함됩니다
판정 기준	<code>replica.lag.time.max.ms</code> 안에 leader 를 따라왔는가	시간 기준입니다. 건수 기준이 아닙니다
빠지는 순간	그 시간 동안 못 따라오면 제외됩니다	장애의 첫 신호입니다
돌아오는 순간	다시 따라잡으면 자동으로 들어옵니다	사람이 넣지 않습니다
지표	ISR 수가 <code>replication.factor</code> 보다 작은 Partition 의 수	<code>UnderReplicatedPartitions</code> 입니다

ISR 에서 빠지는 것 자체는 오류가 아닙니다.

Broker 가 느려지거나 네트워크가 지연돼도 빠집니다. 계속 빠져 있는 것이 문제입니다.

acks

값	언제 응답하는가	유실 범위
0	보내고 바로	Producer 가 응답을 받지 않습니다. Broker 가 정지해도 알지 못합니다
1	leader 가 자기 로그에 쓴 뒤	leader 가 복제 전에 정지하면 그 메시지가 사라집니다
all	ISR 전체가 받은 뒤	ISR 이 전부 정지하지 않는 한 남습니다

acks 는 Producer 가 정하고 **min.insync.replicas** 는 Topic 이 정합니다.
둘이 짝을 이뤄야 내구성이 성립합니다. 한쪽만으로는 의미가 없습니다.

min.insync.replicas 와 acks=all

ISR 수	min.insync.replicas	produce 결과
3	2	성공. 세 곳이 받습니다
2	2	성공. 두 곳이 받습니다
1	2	실패. NotEnoughReplicas
3	3	성공. 세 곳이 다 받아야 합니다
2	3	실패. Broker 1대만 빠져도 멈춥니다

min.insync.replicas 는 「몇 곳이 받아야 성공으로 볼 것인가」입니다.
이 값에 못 미치면 Broker 가 쓰기를 거부합니다. 조용히 덜 복제되는 것보다 낫습니다.

세 값의 조합

조합	무엇을 얻는가	결과
<code>RF=3</code> , <code>acks=1</code>	복제본은 셋인데 leader 만 기다립니다	RF 가 의미 없습니다. leader 가 정지하면 유실
<code>RF=3</code> , <code>acks=all</code> , <code>min.insync=1</code>	ISR 이 1이어도 성공합니다	Broker 둘이 빠져도 계속 씁니다. 유실 위험이 큼니다
<code>RF=3</code> , <code>acks=all</code> , <code>min.insync=2</code>	두 곳 이상이 받아야 성공	가용성과 내구성의 균형점. 이 실습에서 맞출 값
<code>RF=3</code> , <code>acks=all</code> , <code>min.insync=3</code>	세 곳이 다 받아야 성공	가장 안전하지만 1대만 빠져도 멈춥니다

`RF` 만 3으로 두고 `acks=1` 이면 복제본은 백업일 뿐 내구성이 아닙니다. 가장 흔한 오설정입니다.

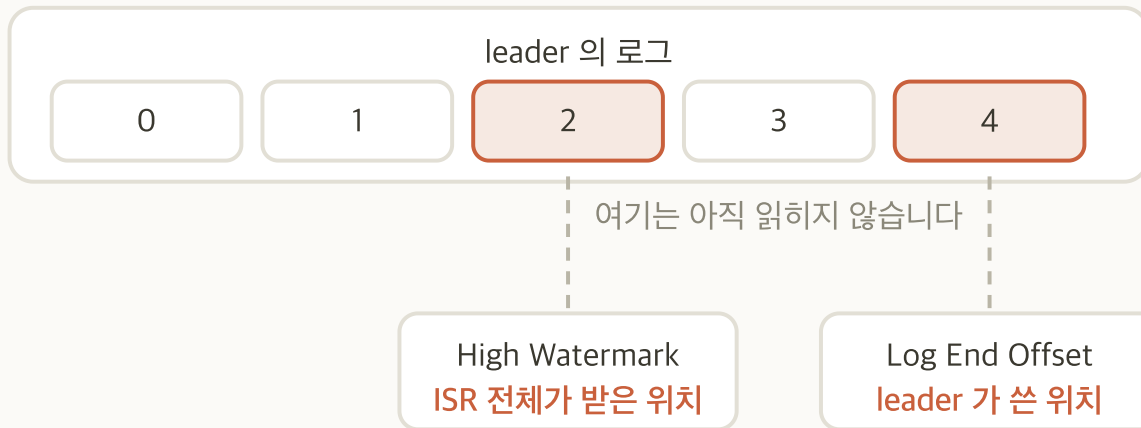
unclean.leader.election.enable

값	무엇이 일어나는가	대가
false (기본)	ISR 안의 복제본만 leader 가 됩니다	ISR 이 비면 그 Partition 이 멈춥니다. 데이터는 지킵니다
true	ISR 밖의 뒤쳐진 복제본도 leader 가 됩니다	Partition 이 다시 동작합니다. 뒤쳐진 만큼의 메시지가 사라집니다

가용성과 데이터를 맞바꾸는 설정입니다.

둘 다 얻는 설정이 없습니다. 무엇을 감수할지 고르는 것이 운영 판단입니다.

High Watermark 와 Log End Offset



- **LEO**: leader 가 자기 로그에 쓴 마지막 위치입니다
- **HW**: ISR 전체가 받은 위치입니다. **Consumer** 는 여기까지만 읽습니다
- **Leader Epoch**: leader 가 바뀐 세대 번호입니다. 교체 후 어디까지가 유효한지 가릅니다

쓴 직후 바로 읽히지 않는 이유가 이것입니다.

복제가 따라와 HW 가 올라가야 Consumer 에게 보입니다. 되돌려질 수 있는 것을 보여 주지 않기 위해서입니다.

Broker 정지와 produce

상황	ISR	produce
정상	3	성공
Broker 1대 정지	2	성공. <code>min.insync.replicas=2</code> 를 만족합니다
Broker 2대 정지	1	실패. <code>NotEnoughReplicas</code>

Broker 가 3대여야 이 관찰이 성립합니다.

2대면 1대만 정지해도 곧바로 produce 가 실패해 볼 구간이 없습니다.

acks 와 min.insync.replicas 확인

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders | grep Isr` 로 모든 Partition 의 `Isr` 이 3인지 확인
- 2 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=2`
- 3 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --command-property acks=1` 로 한 줄, 이어서 `acks=all` 로 한 줄 produce
- 4 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=3` 후 `acks=all` 로 다시 produce. **ISR 이 3이므로 성공합니다**
- 5 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=2` 로 되돌리고 `--describe` 로 확인

마지막 단계를 빠뜨리면 챕터 12에서 Broker 1대를 정지시켰을 때 produce 가 실패합니다.
 앞 장 표의 「Broker 1대 정지에서 성공」이 성립하지 않습니다. 반드시 2로 되돌립니다.

kafka-configs.sh --describe 출력

```
Dynamic configs for topic lab.orders are:
  min.insync.replicas=2 sensitive=false
  synonyms={DYNAMIC_TOPIC_CONFIG:min.insync.replicas=2,
    DYNAMIC_DEFAULT_BROKER_CONFIG:...=1, DEFAULT_CONFIG:...=1}
```

- **DYNAMIC_TOPIC_CONFIG** : Topic 에 걸어 둔 값입니다. 이 값이 우선합니다
- **DYNAMIC_DEFAULT_BROKER_CONFIG** · **DEFAULT_CONFIG** : Broker 에 동적으로 건 값과 Broker 기본값입니다. 여기서는 둘 다 1입니다
- **synonyms** : 같은 설정의 출처를 우선순위 순으로 보여 줍니다

Broker 기본값을 고치고 Topic 설정을 보면 바뀌지 않은 것처럼 보입니다.

Topic 에 값이 걸려 있으면 그 값이 우선합니다. 어느 층의 값을 보고 있는지 먼저 가립니다.

11

Retention 과 segment

시간 기준

용량 기준

segment 롤링

활성 segment

delete 와 compact

디스크 압박

Retention 의 두 기준

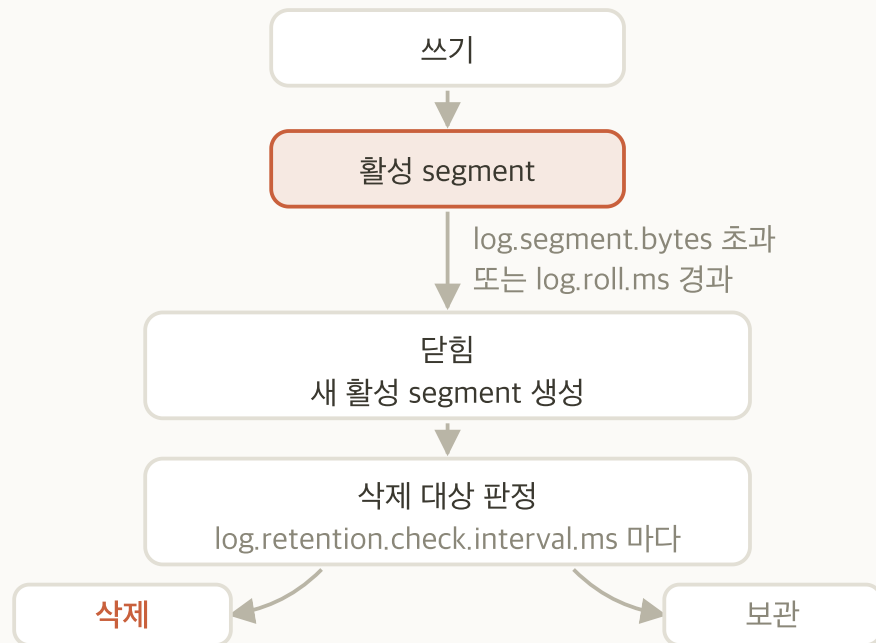
Retention 은 Broker 가 메시지를 언제까지 디스크에 남겨 둘지 정하는 규칙입니다. Consumer 가 읽었는지와 무관하게, 정해진 기준을 넘긴 segment 를 Broker 가 스스로 지웁니다.

설정	기준	걸지 않으면
<code>retention.ms</code>	시간. segment 의 마지막 메시지 시각 기준	기본 7일입니다
<code>retention.bytes</code>	Partition 당 용량	기본이 무제한입니다. 걸지 않으면 디스크가 찹니다
둘 다 걸면	먼저 도달하는 쪽이 적용됩니다	어느 쪽이 걸렸는지 로그로 확인합니다

`retention.bytes` 는 Partition 당입니다.

Partition 6개에 `replication.factor` 3을 곱하면 복제본이 18개이고, 이것이 Broker 3대에 나뉘어 한 대에 여섯 배가 쌓입니다. 클러스터 전체로는 열여덟 배입니다.

segment 롤링과 삭제



- 쓰기가 계속되는 동안에는 활성 segment 가 삭제 대상이 아닙니다: 닫혀야 판정에 들어갑니다
- 판정 주기: `log.retention.check.interval.ms` 마다입니다. 제품 기본값은 5분이고 이 실습은 1분으로 낮춰 두었습니다
- 그래서 즉시 지워지지 않습니다: 롤링과 주기 둘을 기다려야 합니다

Retention 을 줄여도 디스크가 곧바로 비지 않습니다.

적재가 계속되면 활성 segment 는 남습니다. 적재가 멈추면 활성 segment 도 닫혀 삭제 대상이 되고, 새 빈 segment 가 하나 열립니다.

cleanup.policy

값	무엇을 남기는가	쓰는 곳
delete (기본)	오래된 segment 부터 지웁니다	이벤트 스트림. 지나간 것은 필요 없습니다
compact	키별 최신 값만 남깁니다	상태 저장. <code>__consumer_offsets</code> 가 이 방식입니다
compact,delete	압축하고 오래된 것도 지웁니다	상태를 두되 무한히 보관하지는 않을 때

compact 는 키가 있어야 성립합니다.

키가 없는 메시지는 무엇의 최신 값인지 알 수 없습니다.

디스크 포화의 영향

- Broker 가 쓰기를 실패합니다: 그 Broker 의 Partition 이 전부 멈춥니다
- ISR 이 무너집니다: follower 가 복제를 못 받으면 ISR 에서 빠집니다
- Retention 을 줄여도 곧바로 반영되지 않습니다: 롤링과 판정 주기를 기다려야 합니다
- 그래서 미리 설정합니다: `retention.bytes` 를 처음부터 걸어 둡니다

디스크가 차기 시작한 뒤에 Retention 을 줄이는 것은 늦습니다.
지워질 때까지 시간이 걸리고 그동안 쓰기는 계속 실패합니다.

segment 생성

- 1 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config segment.bytes=1048576` 로 segment 를 1 MiB 로
- 2 **Consumer 두 터미널을 먼저 정지.** 정지하지 않으면 10만 건이 화면에 그대로 출력됩니다
- 3 `kafka-producer-perf-test.sh --topic lab.orders --num-records 100000 --record-size 200 --throughput -1 --command-property bootstrap.servers=10.0.1.5:9092` 로 적재
- 4 `ansible kafka -a "ls -l /var/lib/kafka/lab.orders-0"` 로 segment 파일 목록 확인
- 5 **파일 이름과 개수를 기록**

완료 판정

`.log` 파일이 여러 개 보이면 완료입니다.

Partition 디렉터리

```
-rw-r--r-- 1048274 00000000000000000000.log
-rw-r--r-- 1048128 000000000000000004994.log
-rw-r--r-- 1048128 000000000000000009986.log
-rw-r--r-- 294786 000000000000000014978.log <- 활성
-rw-r--r-- 10485760 000000000000000014978.index
... segment 마다 .index · .timeindex · .snapshot 이 함께 있습니다
```

- 파일 이름은 첫 Offset 입니다: `000000000000000004994.log` 는 4994번부터 담습니다
- 닫힌 segment 는 `segment.bytes` 에 가깝습니다: 활성 segment 만 작습니다
- `.index` · `.timeindex` 가 함께 있습니다: 활성 segment 의 인덱스는 미리 크게 잡혀 있습니다
- 디렉터리에는 이 밖에도 있습니다: `leader-epoch-checkpoint` 와 `partition.metadata` 가 함께 놓입니다

Retention 조정과 삭제 확인

- 1 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config retention.ms=60000` 으로 Retention 을 1분으로
- 2 곧바로 `ansible kafka -a "ls /var/lib/kafka/lab.orders-0"` 실행. 파일이 그대로인 것 확인
- 3 판정 주기가 지난 뒤 같은 명령을 다시 실행. **최대 2분입니다**
- 4 오래된 segment 부터 사라졌는지, 새 빈 segment 하나만 남았는지 확인
- 5 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config retention.bytes=10485760` 을 걸고 다시 확인
- 6 Consumer 는 p.106 에서 이미 정지했습니다. `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --group lab-cg --topic lab.orders --reset-offsets --to-earliest --dry-run` 으로 `NEW-OFFSET` 이 0이 아닌 것 확인

여기서 거는 Retention 은 이미 쌓인 segment 를 영구 삭제합니다.

삭제된 것은 복구할 수 없고, 설정을 되돌리기 전까지 이후 적재분도 계속 지워집니다. 남길 출력은 먼저 기록해 두세요.

삭제 전후 대비

```
# retention.ms 조정 직후. .log 가 넷
...000000.log ...004994.log ...009986.log ...014978.log
# 판정 주기가 지난 뒤. 넷 다 .deleted 로 바뀌고
...016382.log <- 새로 열린 빈 segment 하나만 남습니다
```

- 오래된 것부터 사라집니다: 앞 Offset 의 파일이 먼저 지워집니다
- 적재가 멈추면 활성 segment 도 닫히고 지워집니다: 새 빈 segment 하나만 남습니다
- 조정 직후에는 그대로입니다: 판정 주기를 기다려야 합니다

--to-earliest 가 가리키는 위치가 이 결과로 달라집니다.

앞부분이 지워졌으므로 「남아 있는 가장 오래된 위치」가 0이 아닙니다.

12

Broker 정지와 복귀

ISR 축소

leader 교체

produce 계속

quorum 유지

복귀

리소스 정리

이 실습의 범위

- **미리 알려 주고 시작합니다:** 무엇을 정지시킬지와 무엇이 일어나야 하는지를 먼저 안내합니다
- **원인을 찾는 실습이 아닙니다:** 구조가 문서대로 동작하는지 관찰합니다
- **Broker 1대만 정지시킵니다:** 2대를 정지시키면 produce 가 실패합니다

시작 전에 `lab.orders` 의 `min.insync.replicas` 가 2인지 확인합니다.

3인 상태에서 시작하면 Broker 1대만 정지시켜도 produce 가 실패합니다. 챕터 10 실습의 마지막 단계에서 2로 되돌렸는지 먼저 봅니다.

Broker 1대 정지

단계	예상 반응	확인 방법	판정
<code>ansible <Leader 가 여러 개인 Broker> -m systemd -a "name=kafka state=stopped" --become</code>	그 Broker 가 ISR 에서 제외	<code>kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders</code>	Isr 이 3개에서 2개로
정지 유지	그 Broker 가 leader 였던 Partition 의 leader 교체	위와 동일	Leader 가 남은 Broker 로 변경
<code>kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --command-property acks=all</code>	성공	Producer 출력	ISR 2개가 <code>min.insync.replicas=2</code> 를 만족
controller 상태 확인	quorum 이 과반 2로 유지	<code>kafka-metadata-quorum.sh --bootstrap-server 10.0.1.5:9092 describe --status</code>	LeaderId 가 여전히 하나
지표 확인	복제 부족 Partition 발생	Grafana 의 <code>UnderReplicatedPartitions</code>	0이 아닌 값

측정값: 정지부터 ISR 에서 빠질 때까지 6초에서 11초였습니다. 정상 종료라 `replica.lag.time.max.ms` 30초를 기다리지 않습니다. `exporter` 가 Broker JVM 안에 있어 `kafka_jmx` 대상도 끝납니다.

ISR 이 줄어든 상태

```
Topic: lab.orders PartitionCount: 6 ReplicationFactor: 3 Configs: min.insync.replicas=2,...
Topic: lab.orders Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
Topic: lab.orders Partition: 1 Leader: 3 Replicas: 2,3,1 Isr: 3,1
... Partition 2,3,4,5 도 Isr 에서 2가 빠졌습니다
```

- **Replicas** 는 그대로입니다: 복제본을 둘 자리는 여전히 셋입니다
- **Isr** 에서 2가 빠졌습니다: 따라오지 못하는 복제본입니다
- 일부 Partition 의 **Leader** 가 바뀌었습니다: 2가 leader 였던 Partition 입니다

Replicas 와 **Isr** 이 갈리는 것이 이 실습의 관찰 대상입니다.
정상 상태에서는 둘이 같습니다.

개념과 관찰 결과의 대응

개념	관찰 결과
따라잡은 복제본의 집합인 ISR	<code>Isr</code> 에서 한 Broker 가 빠졌습니다
Partition 마다 따로 있는 leader	정지한 Broker 가 맡던 Partition 만 leader 가 바뀌었습니다
<code>min.insync.replicas=2</code> 에서 1대 정지 시 쓰기 유지	<code>acks=all</code> produce 가 성공했습니다
과반이 2인 quorum	<code>LeaderId</code> 가 유지됐습니다
복귀 뒤에도 유지되는 leader 배치	복귀 뒤에도 배치가 달랐습니다

리소스 정리

```
# 자기 PC 의 lab/terraform/03-kafka 에서 실행. ops 노드 아님
terraform destroy -var-file=session.tfvars
```

- 포털에서 리소스 그룹이 실제로 사라졌는지 확인합니다. 한 번에 끝나지 않는 경우가 있습니다
- 세션 간에 자원을 넘기지 않습니다. 다음 세션은 04-operation 을 새로 만듭니다
- 다음 세션은 Redis·MongoDB·Kafka 를 모두 새로 띄웁니다. 이 세션의 Kafka 를 남기지 않습니다

destroy 는 리소스 그룹과 그 안의 자원을 전부 삭제합니다.

되돌릴 수 없습니다. **describe** 출력과 설정값 기록 중 남길 것을 먼저 자기 PC 로 옮기세요.